МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»

Кафедра комп’ютерних наук

КУРСОВА РОБОТА
(ПОЯСНЮВАЛЬНА ЗАПИСКА)

з дисципліни «Алгоритми розв’язування задач у технічних науках»
на тему:
«АЛГОРИТМИ ВИДАЧІ КОНТЕНТУ КОРИСТУВАЧУ»

студента II курсу групи КНм-25-1
спеціальності 122 «Комп’ютерні науки»
Розбицького Романа Едуардовича
(прізвище, ім’я та по-батькові)

Керівник: Праздніков О.В.
_____

Дата захисту: "17" жовтня 2026 р.
Національна шкала _____
Кількість балів: _____
Оцінка: ECTS _____

Члени комісії _____ Праздніков О.В.
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

Житомир – 2026

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»

Факультет інформаційно-комп’ютерних технологій
Кафедра комп’ютерних наук
Освітній рівень: бакалавр
Спеціальність 122 «Комп'ютерні науки»

«ЗАТВЕРДЖУЮ»
Зав. кафедри
_____Віталій Левківський
“____”_____2026 р.

ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТУ
Розбицькому Роману Едуардовичу

1. Тема роботи: «Оптимізація складу та послідовності виплавки сплавів у електродуговій печі»
керівник курсового проекту: Празднiков Олександр____
2. Строк подання студентом: “17” жовтня 2026 р.
3. Вхідні дані до роботи: відсутні
4. Зміст розрахунково-пояснювальної записки (перелік питань які підлягають розробці)
   - аналіз предметної області;
   - математична постановка;
   - проєктування та реалізація коду мовою Javascript;
   - аналіз результатів роботи алгоритмів;
5. Перелік графічного матеріалу(з точним зазначенням обов’язкових креслень)
   1) Посилання на репозиторій:
6. Консультанти розділів проекту (роботи)

Розділ	Прізвище, ініціали та посади консультанта	Підпис, дата	
		завдання видав	завдання прийняв
1, 2, 3	Празднiков В.В., а. кафедри каф. КН	03.10	03.10

7. Дата видачі завдання “3” Жовтня 2026 р.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів курсового проекту	Строк виконання етапів проекту	Примітки
1	Вибір і затвердження теми, отримання завдання, підбір і вивчення літератури	03.10 – 04.10	виконано
2	Аналіз предметної області, класифікація алгоритмів видачі контенту, вибір алгоритмів для дослідження	05.10 – 06.10	виконано
3	Вивчення теоретичних основ обраних алгоритмів і методів оцінювання якості, написання розділу 1	07.10 – 08.10	виконано
4	Проєктування архітектури програмного застосунку, розробка моделі синтетичних даних	09.10 – 10.10	виконано
5	Програмна реалізація алгоритмів видачі контенту та модуля оцінювання мовою JavaScript	11.10 – 12.10	виконано
6	Тестування застосунку, проведення експериментів, дослідження впливу параметрів і стійкості результатів	13.10	виконано
7	Аналіз результатів, обґрунтування вибору алгоритму, написання розділів 2 і 3	14.10	виконано
8	Написання вступу і висновків, оформлення пояснювальної записки, списку джерел і додатка	15.10	виконано
9	Фіналізація звіту	15.10 – 16.10	виконано

Студент _____ Розбицький Р.Е.
(підпис) (прізвище та ініціали)

Керівник проекту _____ Празднiков О.В.
(підпис) (прізвище та ініціали)

ЗМІСТ

ВСТУП 5
РОЗДІЛ 1. ТЕОРЕТИЧНІ ОСНОВИ АЛГОРИТМІВ ВИДАЧІ КОНТЕНТУ ... 7
1.1. Задача видачі контенту 7
1.2. Формальна постановка задачі 8
1.3. Класифікація алгоритмів 8
1.4. Неперсоналізовані алгоритми 9
1.5. Персоналізовані алгоритми 12
1.6. Проблема дослідження і використання 14
1.7. Методи оцінювання 15
1.8. Висновки до розділу 1 16
РОЗДІЛ 2. ПРАКТИЧНА РЕАЛІЗАЦІЯ АЛГОРИТМІВ ВИДАЧІ КОНТЕНТУ 17
2.1. Засоби реалізації та архітектура застосунку 17
2.2. Генерація синтетичного набору даних 18
2.3. Реалізація неперсоналізованих алгоритмів 20
2.4. Реалізація персоналізованих алгоритмів 21
2.5. Оцінювання та консольний інтерфейс 25
2.6. Висновки до розділу 2 26
РОЗДІЛ 3. ПОРІВНЯЛЬНИЙ АНАЛІЗ ТА ВИБІР АЛГОРИТМУ 28
3.1. Умови експерименту та основні результати 28
3.2. Аналіз за метриками точності 29
3.3. Аналіз за метриками різноманітності, новизни та свіжості 30
3.4. Вплив параметрів алгоритмів 31
3.5. Стійкість результатів 32
3.6. Обґрунтування вибору алгоритму 34
3.7. Висновки до розділу 3 36
ВИСНОВКИ 37
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 39
ДОДАТОК А. ЛІСТИНГ ПРОГРАМНОГО ЗАСТОСУНКУ 42

ВСТУП

Актуальність теми. Обсяг контенту в сучасних інформаційних системах у тисячі разів перевищує можливості людини його переглянути: щохвилини на відеохостинги завантажуються сотні годин відео, у соціальних мережах щосекунди з'являються тисячі публікацій, інтернет-магазини пропонують мільйони товарів. За таких умов вирішальним стає не наявність контенту, а те, яку його частину побачить конкретний користувач і в якому порядку. Це завдання розв'язують алгоритми видачі контенту, або рекомендаційні системи.

Від якості алгоритму видачі безпосередньо залежать ключові показники цифрового продукту: час, проведений у застосунку, кількість переглядів, конверсія, утримання аудиторії. За даними інженерів Netflix [11], рекомендаційна система впливає на вибір приблизно 80% годин перегляду на платформі, а її ділова цінність перевищує мільярд доларів на рік. Водночас алгоритмам видачі закидають створення «бульбашок фільтрів» [14], концентрацію уваги на невеликій кількості популярних авторів і непрозорість, тому вибір алгоритму є не лише технічним, а й етичним рішенням.

Існує велика кількість алгоритмів видачі — від хронологічного впорядкування до нейромережевих моделей. Для розробника невеликого продукту постає практичне питання: який алгоритм забезпечить достатню якість без надмірних витрат на реалізацію? Відповідь потребує систематичного порівняння алгоритмів в однакових умовах за об'єктивними метриками, що й зумовлює актуальність теми.

Мета роботи — дослідити основні класи алгоритмів видачі контенту, реалізувати їх мовою JavaScript, експериментально порівняти та обґрунтувати вибір алгоритму з найкращим співвідношенням якості видачі та складності реалізації.

Для досягнення мети поставлено такі **завдання**:

1) сформулювати задачу видачі контенту як задачу ранжування та класифікувати алгоритми її розв'язання;
2) розглянути теоретичні основи обраних алгоритмів і методи оцінювання їхньої якості;
3) розробити модель синтетичних даних і реалізувати вісім алгоритмів у вигляді модульного консольного застосунку на платформі Node.js;
4) провести експериментальне порівняння алгоритмів, дослідити вплив їхніх параметрів і стійкість результатів;
5) обґрунтувати вибір найкращого алгоритму з урахуванням складності реалізації.

Об'єктом дослідження є процес формування стрічки контенту в інформаційних системах.

Предметом дослідження є алгоритми ранжування контенту: хронологічний, випадковий, за популярністю, з часовим згасанням популярності, контентна фільтрація, колаборативна фільтрація, гібридна зважена модель та ε-greedy.

Методи дослідження: методи інформаційного пошуку (векторна модель, косинусна подібність, TF-IDF), математичної статистики (інтервали Вілсона, байєсове оцінювання, t-критерій Стьюдента), теорії навчання з підкріпленням (задача багаторукого бандита), імітаційне моделювання й обчислювальний експеримент.

Практичне значення полягає в тому, що розроблений застосунок може слугувати основою модуля рекомендацій вебзастосунку та інструментом для порівняння нових алгоритмів, а отримані результати дозволяють обґрунтовано обрати алгоритм залежно від вимог системи.

Структура роботи. Робота складається зі вступу, трьох розділів, висновків, списку використаних джерел і додатка. У першому розділі розглянуто теоретичні основи алгоритмів видачі та методи їх оцінювання, у другому — програмну реалізацію, у третьому — результати експериментів і обґрунтування вибору алгоритму. Додаток містить повний лістинг програми.

РОЗДІЛ 1. ТЕОРЕТИЧНІ ОСНОВИ АЛГОРИТМІВ ВИДАЧІ КОНТЕНТУ

1.1. Задача видачі контенту

Будь-яка інформаційна система, що містить більше контенту, ніж користувач може переглянути за один сеанс, мусить вирішувати, що саме показати і в якій послідовності. Пошукові системи відповідають на явний запит користувача, а рекомендаційні системи пропонують контент без запиту, спираючись на відомості про користувача та контент. Алгоритми видачі, що розглядаються в роботі, належать до другого класу: стрічка новин соціальної мережі, головна сторінка відеохостингу, добірка статей на новинному сайті, блок «Рекомендовано для вас» в онлайн-кінотеатрі.

Попри різноманітність предметних областей, ці задачі мають спільну структуру. Є множина користувачів, множина об'єктів контенту та історія взаємодій між ними (перегляди, вподобання, покупки). Для кожного користувача потрібно відібрати невелику кількість об'єктів і впорядкувати їх так, щоб найцікавіші опинилися вгорі. Обмеженість простору показу є принциповою: екран смартфона вміщує 3–5 публікацій, тому якість верхньої частини стрічки визначає сприйняття всієї системи.

Задача видачі виникає в багатьох предметних областях. У соціальних мережах це впорядкування публікацій друзів і спільнот; у медіа — добір статей з урахуванням актуальності та інтересів читача; в електронній комерції — показ товарів, які користувач найімовірніше придбає; у стрімінгових сервісах — пропозиція фільмів і музики; на форумах на кшталт Reddit чи Hacker News — формування головної сторінки з найбільш обговорюваних тем. У промислових системах з мільйонами об'єктів застосовують двоетапну архітектуру: швидкий алгоритм відбирає кілька сотень кандидатів, а складніша модель ранжує лише їх [4].

«Цікавість» контенту не є єдиною ціллю. Власник платформи зацікавлений і в тому, щоб стрічка була свіжою, тематично різноманітною, а

нові автори отримували аудиторію. Ці цілі часто суперечать одна одній, тому в роботі алгоритми оцінюються за системою метрик, а не за однією.

1.2. Формальна постановка задачі

Нехай U — множина з m користувачів, I — множина з n об'єктів (публікацій). Кожен об'єкт i описується множиною тегів T(i), віком age(i), автором і лічильниками вподобань likes(i) та переглядів views(i). Історію взаємодій подамо бінарною матрицею R розміром m × n, де r(u, i) = 1, якщо користувач u вподобав об'єкт i. Такий зворотний зв'язок називають неявним (implicit feedback), на відміну від явних оцінок за шкалою.

Позначимо через H(u) історію користувача u, а через C(u) = I \ H(u) — множину кандидатів, тобто об'єктів, яких він ще не бачив. Алгоритм видачі задається функцією оцінки

$$score: U \times I \to \mathbb{R}, \tag{1.1}$$

а стрічка довжини K — це K кандидатів з найбільшими оцінками:

$$Feed(u) = top\text{-}K \{ i \in C(u) \} \text{ за спаданням } score(u, i). \tag{1.2}$$

Отже, задача видачі зводиться до задачі ранжування, а алгоритми відрізняються лише способом обчислення score. Таке уніфіковане формулювання дозволяє винести спільні кроки (формування кандидатів, сортування, відбір перших K) в окремий модуль і порівнювати алгоритми в однакових умовах. Наївна реалізація потребує O(n · c + n log n) операцій на користувача, де c — вартість однієї оцінки. Деякі алгоритми (підвищення різноманітності, дослідження) додатково переранжовують результат з урахуванням уже вибраних об'єктів.

1.3. Класифікація алгоритмів

За ступенем персоналізації алгоритми поділяють на два класи [1; 7; 15]. **Неперсоналізовані** алгоритми формують однакову стрічку для всіх: score(u, i) = f(i). Вони прості, швидкі, прозорі й не мають проблеми «холодного старту»

для нових користувачів, але не враховують індивідуальних інтересів. **Персоналізовані** алгоритми враховують інформацію про конкретного користувача. Основними підходами є контентна фільтрація (рекомендує об'єкти, схожі за змістом на вже вподобані), колаборативна фільтрація (рекомендує те, що вподобали користувачі зі схожою поведінкою) і гібридні системи, що поєднують кілька підходів. Окрему групу складають алгоритми, що враховують компроміс між дослідженням і використанням.

Алгоритми також відрізняються типом сигналів: релевантність (відповідність інтересам), якість (сприйняття аудиторією), свіжість і різноманітність. Обрані для дослідження алгоритми охоплюють усі класи і типи сигналів (таблиця 1.1).

Таблиця 1.1 – Алгоритми, обрані для дослідження

Алгоритм	**Клас**	**Основний сигнал**
Хронологічний	неперсоналізований	свіжість
Випадковий	базова лінія	—
За популярністю	неперсоналізований	якість
Hacker News / Reddit	неперсоналізований	якість + свіжість
Контентна фільтрація	персоналізований	релевантність за змістом
Колаборативна (kNN)	персоналізований	релевантність за поведінкою
Гібридна зважена	гібридний	усі сигнали + різноманітність
ε-greedy	дослідження/використання	релевантність + дослідження

1.4. Неперсоналізовані алгоритми

1.4.1. Хронологічна стрічка

Найпростіший спосіб видачі — показувати найновіші об'єкти першими:

$$score(u, i) = -age(i). \tag{1.3}$$

Хронологічну стрічку використовували ранні соціальні мережі, і вона досі є основною в месенджерах і стрічках підписок. Її головні переваги — повна прозорість (користувач розуміє, чому бачить публікацію), мінімальна обчислювальна складність, відсутність потреби в історії та «справедливість»

щодо авторів. Недоліки — ігнорування інтересів і якості, а також вразливість до авторів, які публікують надто часто й витісняють усіх інших. Для пом'якшення останнього застосовують обмеження кількості публікацій одного автора на сторінці. Для підвантаження стрічки використовують курсорну пагінацію: клієнт передає час останньої показаної публікації, і сервер повертає старіші. На відміну від пагінації за номером сторінки, це стійко до появи нових публікацій між запитами.

1.4.2. Випадкова видача

Випадкова видача рідко використовується як основний алгоритм, але є необхідною **базовою лінією** експерименту: будь-який «розумний» алгоритм мусить її статистично значуще перевершувати. Очікувана точність випадкової видачі дорівнює частці релевантних об'єктів серед кандидатів. Рівномірна вибірка K об'єктів без повернення реалізується частковим перемішуванням Фішера — Єйтса [9] за O(K) обмінів.

Узагальненням є зважена вибірка, в якій шанс об'єкта пропорційний його вазі w(i), наприклад популярності. Алгоритм A-Res П. Ефраїмідіса та П. Спіракіса [5] генерує для кожного об'єкта випадкове число v(i) ∈ [0, 1) і ключ

$$key(i) = v(i) \wedge (1 / w(i)), \tag{1.4}$$

після чого відбирає K об'єктів з найбільшими ключами. Популярне випадає частіше, але шанс має кожен об'єкт.

1.4.3. Ранжування за популярністю

Об'єкти, що сподобалися багатьом, ймовірно, сподобаються і поточному користувачу, оскільки якість контенту впливає на вподобання всіх. Ключове питання — як виміряти популярність. Найпростіша оцінка score(i) = likes(i) не враховує кількість показів і створює **ефект Метью**: популярне отримує більше показів і закріплюється на вершині. Нормування на перегляди, score(i) = likes(i) / views(i), нестабільне за малих вибірок: один лайк з одного перегляду дає максимальну оцінку.

Байєсове середнє «підтягує» частку до середнього по каталогу $\bar{p}$, доки переглядів мало:

$$score(i) = (likes(i) + C \cdot \bar{p}) / (views(i) + C), \quad (1.5)$$

де C — сила апріорного припущення в кількості «уявних» переглядів.

Нижня межа довірчого інтервалу Вілсона [20], популяризована Е. Міллером [13], дає песимістичну оцінку частки з урахуванням невизначеності:

$$score(i) = (\hat{p} + z^2/2n - z \cdot \sqrt{(\hat{p}(1 - \hat{p})/n + z^2/4n^2)}) / (1 + z^2/n), \quad (1.6)$$

де $\hat{p}$ = likes(i) / views(i), n = views(i), z = 1,96 для довіри 95%. Для одного лайка з одного перегляду оцінка становить лише 0,21, а для 900 лайків з 1000 переглядів — 0,88, тобто формула автоматично «карає» ненадійні оцінки. Спільний недолік усіх варіантів — ігнорування свіжості: стрічка «застигає».

1.4.4. Популярність з часовим згасанням

Формули з часовим згасанням поєднують популярність і свіжість. Сайт Hacker News використовує степеневе згасання:

$$score(i) = (P(i) - 1) / (T(i) + 2) \wedge G, \quad (1.7)$$

де P — кількість голосів, T — вік у годинах, G = 1,8 — «гравітація». Що більше G, то швидше тонуть старі публікації: при G = 1,8 публікації віком 24 години потрібно приблизно в 49 разів більше голосів, ніж годинній, щоб стояти поруч.

Reddit у режимі «Hot» використовує логарифм популярності та лінійний за часом бонус [16]:

$$score(i) = sign(s) \cdot \log_{10}(\max(|s|, 1)) - T_sec(i) / 45000, \quad (1.8)$$

де s — різниця позитивних і негативних голосів, T_sec — вік у секундах. Логарифм зменшує перевагу «хітів»: перші 10 голосів важать стільки ж, скільки наступні 90. Кожні 12,5 години віку еквівалентні десятикратному зменшенню кількості голосів. Обидві формули добре визначають, що актуально для всіх, але не що цікаво конкретному користувачу.

1.5. Персоналізовані алгоритми

1.5.1. Контентна фільтрація

Контентна фільтрація рекомендує об'єкти, схожі на вже вподобані користувачем. Її основою є векторна модель інформаційного пошуку [17]: кожен об'єкт подається вектором у просторі тегів. За **бінарного** зважування координата дорівнює 1, якщо об'єкт має тег. За зважування **TF-IDF**

$$x(i)_t = tf(t, i) \cdot ln(n / df(t)), \tag{1.9}$$

де tf = 1 / |T(i)| — частка тегу серед тегів об'єкта, df(t) — кількість об'єктів з тегом t. Множник IDF підвищує вагу рідкісних тегів, множник TF зменшує вагу тегів у багатотемних об'єктах.

Профіль користувача — сума векторів вподобаних об'єктів, p(u) = Σ x(i) по i ∈ H(u). Оцінка кандидата — **косинусна подібність** його вектора до профілю:

$$cos(p, x) = \Sigma_t\, p_t\, x_t / (\sqrt{(\Sigma_t\, p_t^2)} \cdot \sqrt{(\Sigma_t\, x_t^2)}). \tag{1.10}$$

Косинус не залежить від довжин векторів, а лише від «пропорцій» інтересів, і для невід'ємних векторів лежить у [0, 1]. Оскільки чисельник є сумою внесків окремих тегів, рекомендацію легко пояснити («бо ви цікавитеся подорожами»).

Переваги методу: незалежність від інших користувачів, відсутність холодного старту для нових об'єктів, пояснюваність, простота і лінійна складність. Недоліки: **надмірна спеціалізація**, що веде до «бульбашки фільтрів» [14]; залежність від якості опису об'єктів; холодний старт для нових користувачів; ігнорування якості й свіжості.

1.5.2. Колаборативна фільтрація

Колаборативна фільтрація рекомендує те, що вподобали схожі користувачі, і не потребує опису контенту, тому застосовна до будь-яких

об'єктів і здатна виявляти неочевидні зв'язки [12; 18]. Класичний алгоритм користувацьких найближчих сусідів (user-based kNN) [6] має три кроки.

Спочатку обчислюється подібність цільового користувача до кожного іншого. Для множин вподобань природною мірою є **коефіцієнт Жаккара**:

$$J(u, v) = |H(u) \cap H(v)| / |H(u) \cup H(v)|. \quad (1.11)$$

Альтернативою є косинус зі зважуванням за оберненою частотою користувачів iuf(i) = ln(m / n(i)): спільне вподобання рідкісного об'єкта свідчить про схожість більше, ніж спільне вподобання загальновідомого хіта.

Далі відбирається k найближчих сусідів N_k(u), і оцінка об'єкта обчислюється як нормована сума подібностей сусідів, що його вподобали:

$$score(u, i) = \Sigma_{v \in N_k(u)} sim(u, v) \cdot r(v, i) / \Sigma_{v \in N_k(u)} sim(u, v). \quad (1.12)$$

Поряд з підходом, орієнтованим на користувачів, широко застосовується підхід, орієнтований на об'єкти (item-based) [18]: обчислюється подібність між об'єктами за множинами користувачів, які їх вподобали, і користувачеві пропонуються об'єкти, схожі на вже вподобані ним. Цей варіант стабільніший, коли користувачів значно більше, ніж об'єктів, і дозволяє заздалегідь обчислити таблицю подібностей; саме його використала Amazon для рекомендацій «Клієнти, які купили цей товар, також купили» [12].

Головні недоліки методу — холодний старт для нових користувачів і об'єктів, **розрідженість даних** (перетини історій малі, тому оцінки подібності «шумні»), зміщення до популярного і квадратична за кількістю користувачів складність. Методи на основі моделей, зокрема матрична факторизація [10], частково долають ці проблеми, але потребують навчання і в роботі не реалізуються.

1.5.3. Гібридні моделі

Переваги та недоліки контентного і колаборативного підходів взаємодоповнювальні, тому на практиці їх поєднують. Р. Берк виділив сім способів гібридизації [2], найпоширенішим з яких є **зважений гібрид**:

$$score(u, i) = \Sigma_j w_j \cdot \hat{s}_j(u, i), \quad \Sigma_j w_j = 1, \tag{1.13}$$

де ŝ_j — оцінки окремих сигналів, приведені мінімаксною нормалізацією до [0, 1]:

$$\hat{s}(i) = (s(i) - \min s) / (\max s - \min s). \tag{1.14}$$

Свіжість у гібриді зручно задавати експоненційним згасанням fresh(i) = exp(−ln 2 · age(i) / T½), де T½ — час, за який актуальність падає вдвічі. Для нових користувачів з короткою історією частину ваги персональних сигналів доцільно передавати популярності (адаптивні ваги).

Навіть точний алгоритм схильний заповнювати стрічку однотипними об'єктами, що погіршує сприйняття [24]. Для підвищення різноманітності застосовують переранжування **Maximal Marginal Relevance** [3]: стрічка будується жадібно, на кожному кроці обирається об'єкт з максимальним значенням

$$MMR(i) = \lambda \cdot score(u, i) - (1 - \lambda) \cdot \max_{j \in S} sim(i, j), \tag{1.15}$$

де S — уже вибрані об'єкти, λ ∈ [0, 1] — баланс релевантності та різноманітності.

1.6. Проблема дослідження і використання

Персоналізовані алгоритми показують те, що, згідно з даними, найімовірніше сподобається (використання, exploitation). Але так система ніколи не дізнається про нові інтереси користувача, а нові публікації, що не потрапили на вершину рейтингу, ніколи не отримають показів. Для виходу з цієї петлі потрібне **дослідження** (exploration) — свідомий показ об'єктів з

невизначеною оцінкою. Математичною моделлю цього компромісу є задача про багаторукого бандита [19].

Найпростіша стратегія її розв'язання — **ε-greedy**: з імовірністю 1 − ε обирається найкраща за оцінкою дія, з імовірністю ε — випадкова. Для стрічки це застосовується до кожної позиції окремо. Доцільно зменшувати ε з накопиченням історії користувача n:

$$\varepsilon(n) = \max(\varepsilon_{min}, \varepsilon_0 / (1 + n / \tau)), \quad (1.16)$$

а також спрямовувати дослідження на малопопулярні об'єкти і захищати від нього верхні позиції стрічки. Досконаліші стратегії (UCB, семплювання Томпсона) мають кращі теоретичні гарантії, але складніші в реалізації.

1.7. Методи оцінювання

Найдостовірнішим методом оцінювання є онлайн-експеримент (A/B-тестування), але він доступний лише власникам реальних систем. У роботі використано **офлайн-оцінювання** [25]: частина вподобань кожного користувача приховується (тестова множина T(u)), алгоритм будує стрічку L(u) за рештою даних, і перевіряється збіг стрічки з прихованими вподобаннями. Критично важливо уникнути **витоку даних**: усі статистики, доступні алгоритмам, мають обчислюватися лише за навчальною множиною.

Точність і повнота на K позиціях визначаються як

$$Precision@K = |L(u) \cap T(u)| / K, \quad Recall@K = |L(u) \cap T(u)| / |T(u)|. \quad (1.17)$$

HitRate — частка користувачів, у стрічці яких є хоча б одне влучання. Оскільки верхні позиції стрічки важливіші, основною метрикою обрано **нормалізований дисконтований кумулятивний виграш** [8]:

$$nDCG@K = (1 / IDCG) \cdot \Sigma_{k=1..K} rel(i_k) / \log_2(k + 1), \quad (1.18)$$

де rel(i_k) = 1 для влучання на позиції k, а IDCG — значення суми за ідеального ранжування. Влучання на першій позиції дає внесок 1, на десятій — 0,29.

Метрики «поза точністю» [21] характеризують інші аспекти якості: **покриття** — частка каталогу, що хоч раз потрапила у стрічки; **різноманітність ILD** — середня попарна відстань $1 - J$ між тегами об'єктів стрічки; **новизна** — середня самоінформація $-\log_2 p(i)$ показаних об'єктів, де $p(i)$ — частка користувачів, які їх вподобали; **середній вік** показаних об'єктів. Додатково враховуються час обчислення й експертна оцінка складності реалізації за п'ятибальною шкалою.

1.8. Висновки до розділу 1

Задача видачі контенту зведена до задачі ранжування: побудови функції оцінки score(u, i) і відбору K об'єктів з найбільшими оцінками. Для дослідження обрано вісім алгоритмів, що охоплюють неперсоналізований, персоналізований, гібридний підходи та стратегію дослідження. Теоретичний аналіз показав, що неперсоналізовані алгоритми прості й прозорі, але не враховують інтересів; контентна фільтрація проста й пояснювана, але схильна до надмірної спеціалізації; колаборативна не потребує опису контенту, але чутлива до розрідженості даних; гібридні моделі компенсують недоліки компонентів ціною складності; дослідження жертвує короткостроковою точністю заради навчання системи. Обґрунтовано систему метрик з основною метрикою nDCG@K.

РОЗДІЛ 2. ПРАКТИЧНА РЕАЛІЗАЦІЯ АЛГОРИТМІВ ВИДАЧІ КОНТЕНТУ

2.1. Засоби реалізації та архітектура застосунку

Для реалізації обрано мову **JavaScript** і середовище **Node.js** [22]. JavaScript є основною мовою веброзробки, де найчастіше постає задача видачі контенту, тому алгоритм можна без змін використати як на сервері, так і в браузері. Стандарт ECMAScript [23] надає колекції `Map` і `Set`, функції вищого порядку для масивів і оператор розгортання, завдяки чому код близький до математичних формул розділу 1. Застосунок не використовує сторонніх бібліотек: генератор псевдовипадкових чисел, міри подібності та статистичні оцінки реалізовано власноруч. Модулі організовано за системою CommonJS, для запуску достатньо Node.js версії 14 або новішої.

Застосунок побудовано за модульним принципом (таблиця 2.1): кожен алгоритм міститься в окремому файлі, спільні функції винесено в бібліотечні модулі, а точка входу викликає алгоритми й виводить результати. Загальний обсяг коду становить близько 1000 рядків.

Таблиця 2.1 – Структура програмного застосунку

Файл	**Призначення**
config.js	глобальні параметри експерименту
lib/random.js	детермінований генератор псевдовипадкових чисел
lib/math.js	міри подібності, нормалізація, інтервал Вілсона
lib/ranking.js	спільний механізм ранжування
data/generator.js	генерація синтетичного набору даних
algorithms/01…08-*.js	вісім алгоритмів видачі
algorithms/index.js	реєстр алгоритмів
metrics.js	метрики якості та процедура оцінювання
main.js	консольний інтерфейс

Центральним рішенням є **єдиний інтерфейс алгоритму**. Кожен модуль експортує назву, оцінку складності реалізації `difficulty`, параметри за замовчуванням `defaults` і дві функції: `scoreAll(user, ctx, opts)`

повертає колекцію `Map` з оцінками всіх кандидатів (формула (1.1)), а `recommend(user, ctx, opts)` — готову стрічку з K ідентифікаторів. Розділення цих функцій дозволяє одним алгоритмам використовувати оцінки інших: гібридна модель викликає `scoreAll` контентної та колаборативної фільтрації і поєднує результати. Параметри алгоритму можна змінити під час виклику через `opts`, що використовується в дослідженні параметрів. Контекст `ctx` містить публікації, користувачів і допоміжні статистики; алгоритми його лише читають, тому не впливають один на одного. Щоб додати новий алгоритм, достатньо створити модуль з цим інтерфейсом і дописати його в реєстр.

Спільні для всіх алгоритмів кроки формули (1.2) реалізовано в модулі `lib/ranking.js` (лістинг 2.1). Вторинний ключ сортування за ідентифікатором робить результат детермінованим за однакових оцінок, які трапляються часто.

Лістинг 2.1 – Спільний механізм ранжування (lib/ranking.js)

```
function candidates(user, ctx) {
  return ctx.items.filter(item => !user.train.has(item.id));
}


function rankScores(scores, k = CONFIG.K) {
  return [...scores.entries()]
    .sort((a, b) => b[1] - a[1] || a[0] - b[0])
    .slice(0, k)
    .map(([id]) => id);
}
```

2.2. Генерація синтетичного набору даних

Для експерименту використано **синтетичні дані**, згенеровані за ймовірнісною моделлю. На відміну від реальних, для них достеменно відомі справжні інтереси користувачів і якість публікацій, що дозволяє не лише виміряти метрики, а й пояснити поведінку алгоритмів. Для відтворюваності реалізовано детермінований генератор псевдовипадкових чисел Mulberry32:

на відміну від `Math.random()`, він дозволяє задати зерно, і однакове зерно завжди дає однакові дані.

Генератор створює 300 публікацій і 100 користувачів. Публікація має автора (з 30), від одного до трьох тегів (з 10 тематик), вік від 0 до 336 годин і приховану якість q(i) ∈ [0, 1). Користувач має 2–3 справжні інтереси, які алгоритмам невідомі. Імовірність вподобання (лістинг 2.2) визначається трьома чинниками:

$$P(like \mid u, i) = base(u, i) \cdot (0{,}5 + q(i)) \cdot (0{,}5 + 0{,}5 \cdot (1 - age(i) / 336)), \quad (2.1)$$

де base(u, i) = 0,12 за кожен тег публікації, що входить до інтересів користувача, і 0,01, якщо збігів немає.

Лістинг 2.2 – Модель імовірності вподобання (data/generator.js)

```
function likeProbability(interests, item) {
  const overlap = item.tags.filter(t => interests.has(t)).length;
  let p = overlap > 0 ? 0.12 * overlap : 0.01;
  p *= 0.5 + item.quality;
  p *= 0.5 + 0.5 * (1 - item.ageHours / CONFIG.MAX_AGE_HOURS);
  return p;
}
```

Отже, у даних закладено три «правди»: персональні інтереси, якість і свіжість. Жоден окремий сигнал не є вичерпним, і ідеальний алгоритм мусить враховувати всі три. Згенеровані вподобання кожного користувача перемішуються, 20% з них відкладаються в тестову множину, решта утворює видиму історію. Лічильники вподобань публікацій обчислюються **лише за навчальною множиною**, що запобігає витоку даних. Кількість переглядів моделюється як кількість вподобань, помножена на випадковий коефіцієнт від 4 до 13, плюс 5–60 фонових переглядів. За зерна 42 набір містить 1440 вподобань: у середньому 11,5 у видимій історії та 2,9 у тестовій множині користувача, щільність матриці взаємодій 4,8%.

2.3. Реалізація неперсоналізованих алгоритмів

Хронологічний алгоритм сортує кандидатів за віком і послідовно додає їх у стрічку, пропускаючи публікації автора, який уже має `maxPerAuthor` (за замовчуванням 2) публікацій на сторінці. Лічильники публікацій авторів зберігаються в `Map`. Для пагінації функція `nextCursor` повертає вік останньої показаної публікації, а при запиті наступної сторінки відкидаються не старші за курсор публікації.

Випадкова видача має два режими. Рівномірний реалізовано частковим перемішуванням Фішера — Єйтса. У зваженому режимі для кожного кандидата обчислюється ключ A-Res (1.4) з вагою 1 + likes(i), після чого стрічка формується спільною функцією `rankScores`, тобто зважена вибірка зводиться до звичайної схеми ранжування.

Алгоритм популярності реалізує чотири способи оцінки, що обираються параметром `mode` (лістинг 2.3): кількість вподобань (за замовчуванням), частка вподобань, байєсове середнє (1.5) з C = 50 і апріорною часткою, обчисленою по каталогу, та нижня межа Вілсона (1.6).

Лістинг 2.3 – Чотири способи оцінки популярності (algorithms/03-popularity.js)

```
function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const prior = globalRatio(ctx);


  return scoreCandidates(user, ctx, item => {
    switch (o.mode) {
      case 'ratio':
        return item.views ? item.likes / item.views : 0;
      case 'bayes':
        return (item.likes + o.bayesC * prior) / (item.views + o.bayesC);
      case 'wilson':
        return wilsonLowerBound(item.likes, item.views, o.z);
      case 'likes':
      default:
        return item.likes;
    }
  });
}
```

Часове згасання реалізовано двома функціями. Формула Hacker News (1.7) використовує кількість вподобань без віднімання одиниці, оскільки в

моделі немає автоматичного голосу автора. Формула Reddit (1.8) переводить вік у секунди й ділить на 45 000; логарифм береться від max(|s|, 1), щоб уникнути логарифма нуля.

2.4. Реалізація персоналізованих алгоритмів

Контентна фільтрація (лістинг 2.4) будує вектор публікації функцією `itemVector`: за замовчуванням з бінарними вагами, за параметра `useIdf` — з вагами TF-IDF (1.9). Профіль користувача `buildProfile` підсумовує вектори вподобаних публікацій, а `scoreAll` обчислює косинусну подібність (1.10) кожного кандидата до профілю. Бінарні ваги обрано за результатами експерименту (розділ 3). Додаткова функція `explain` повертає теги, що дали найбільший внесок у скалярний добуток, і використовується для пояснення рекомендацій. Складність алгоритму лінійна за розміром каталогу.

Лістинг 2.4 – Ядро контентної фільтрації (algorithms/05-content-based.js)

```
function itemVector(item, ctx, useIdf) {
  const v = {};
  const tf = 1 / item.tags.length;
  for (const tag of item.tags) v[tag] = useIdf ? tf * idf(tag, ctx) : 1;
  return v;
}


function buildProfile(user, ctx, useIdf = defaults.useIdf) {
  const profile = {};
  for (const id of user.train) {
    const v = itemVector(ctx.items[id], ctx, useIdf);
    for (const tag in v) profile[tag] = (profile[tag] || 0) + v[tag];
  }
  return profile;
}


function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const profile = buildProfile(user, ctx, o.useIdf);
  const scores = new Map();
  for (const item of candidates(user, ctx)) {
    scores.set(item.id, cosine(profile, itemVector(item, ctx, o.useIdf)));
  }
  return scores;
}
```

Колаборативна фільтрація (лістинг 2.5) обчислює подібність цільового користувача до всіх інших за коефіцієнтом Жаккара (1.11) або за

косинусом з IUF-вагами, відкидає користувачів з подібністю нижче порогу і залишає k = 15 найближчих. Оцінка публікації — сума подібностей сусідів, що її вподобали, поділена на суму подібностей усіх сусідів (1.12). Якщо сусідів не знайдено, оцінкою стає нормована популярність — запасний варіант для холодного старту. Складність становить O(m · L) на користувача, де L — довжина історії, що для великої системи вимагало б попереднього обчислення сусідів.

Лістинг 2.5 – Агрегація оцінок сусідів (algorithms/06-collaborative.js)

```
function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const neighbours = findNeighbours(user, ctx, o);
  const scores = new Map();

  if (neighbours.length === 0) {
    for (const item of candidates(user, ctx)) {
      scores.set(item.id, item.likes / ctx.maxLikes);
    }
    return scores;
  }

  const simSum = neighbours.reduce((s, n) => s + n.sim, 0);
  const raw = new Map();
  for (const { user: v, sim } of neighbours) {
    for (const id of v.train) raw.set(id, (raw.get(id) || 0) + sim);
  }
  for (const item of candidates(user, ctx)) {
    scores.set(item.id, (raw.get(item.id) || 0) / simSum);
  }
  return scores;
}
```

Гібридна модель (лістинг 2.6) отримує оцінки контентної фільтрації, колаборативної фільтрації та популярності (у режимі Вілсона), нормалізує кожну мінімаксною нормалізацією (1.14) і поєднує зі свіжістю з періодом напіврозпаду 96 годин. Ваги за замовчуванням: 0,45 для контентного сигналу, по 0,2 для колаборативного і популярності, 0,15 для свіжості. Функція `adaptWeights` для користувачів з історією менше п'яти вподобань пропорційно передає вагу персональних сигналів популярності.

Лістинг 2.6 – Обчислення гібридної оцінки (algorithms/07-hybrid.js)

```
function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts, weights: { ...defaults.weights,
...(opts.weights || {}) } };
```

```
  const w = adaptWeights(user, o);

  const signals = {
    content: minMaxNormalize(contentBased.scoreAll(user, ctx)),
    collaborative: minMaxNormalize(collaborative.scoreAll(user, ctx)),
    popularity: minMaxNormalize(popularity.scoreAll(user, ctx, { mode: 'wilson'
})),
  };

  const scores = new Map();
  for (const item of candidates(user, ctx)) {
    const freshness = Math.exp(-Math.LN2 * item.ageHours / o.halfLifeHours);
    scores.set(item.id,
      w.content * signals.content.get(item.id) +
      w.collaborative * signals.collaborative.get(item.id) +
      w.popularity * signals.popularity.get(item.id) +
      w.freshness * freshness);
  }
  return scores;
}
```

Після обчислення оцінок стрічка переранжовується методом MMR (1.15) з $\lambda = 0{,}85$ і схожістю Жаккара за тегами. Для ефективності MMR застосовується лише до 50 найкращих кандидатів, множини тегів будуються один раз, а максимальна схожість кожного кандидата з уже вибраними оновлюється інкрементально після кожного вибору. Завдяки цьому кожен крок жадібного алгоритму виконується за лінійний від розміру пулу час.

Реалізацію переранжування наведено в лістингу 2.7. На кожному кроці обирається кандидат з найбільшим значенням $\lambda \cdot \text{score} - (1 - \lambda) \cdot \text{redundancy}$, він переноситься з пулу в стрічку, після чого для решти кандидатів обчислюється лише схожість з цією однією новою публікацією, і, якщо вона більша за збережену, значення надлишковості оновлюється. Оскільки оцінки гібрида і коефіцієнт Жаккара лежать у [0, 1], обидва доданки мають порівнянний масштаб, і параметр λ має ясну інтерпретацію. При $\lambda = 1$ функція зводиться до звичайного ранжування, що використовується, коли MMR вимкнено параметром `useMMR`.

Лістинг 2.7 – Переранжування Maximal Marginal Relevance (algorithms/07-hybrid.js)

```
function mmrRerank(scores, ctx, lambda, k = CONFIG.K) {
  const pool = [...scores.entries()]
    .sort((a, b) => b[1] - a[1] || a[0] - b[0])
    .slice(0, k * 5);
```

```
  const tags = new Map(pool.map(([id]) => [id, new Set(ctx.items[id].tags)]));

  const redundancy = new Map(pool.map(([id]) => [id, 0]));
  const selected = [];

  while (selected.length < k && pool.length) {
    let bestIdx = 0, bestValue = -Infinity;
    pool.forEach(([id, score], idx) => {
      const value = lambda * score - (1 - lambda) * redundancy.get(id);
      if (value > bestValue) { bestValue = value; bestIdx = idx; }
    });
    const [chosen] = pool.splice(bestIdx, 1)[0];
    selected.push(chosen);
    for (const [id] of pool) {
      const sim = jaccard(tags.get(id), tags.get(chosen));
      if (sim > redundancy.get(id)) redundancy.set(id, sim);
    }
  }
  return selected;
}
```

Алгоритм ε-greedy (лістинг 2.8) будує персональну стрічку гібридної моделі довжиною 3K, формує пул дослідження з менш популярної половини решти кандидатів і заповнює стрічку позиція за позицією. Перші дві позиції завжди беруться з персональної стрічки, на решті з імовірністю ε обирається випадкова публікація з пулу дослідження. Значення ε згасає за формулою (1.16) з $\varepsilon_0 = 0{,}2$, $\varepsilon_{min} = 0{,}05$, $\tau = 40$: для середнього користувача з 14 вподобаннями $\varepsilon \approx 0{,}15$.

Лістинг 2.8 – Формування стрічки стратегією ε-greedy (algorithms/08-epsilon-greedy.js)

```
function recommend(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const eps = currentEpsilon(user, o);

  const scores = hybrid.scoreAll(user, ctx);
  const exploit = hybrid.mmrRerank(scores, ctx, hybrid.defaults.mmrLambda,
CONFIG.K * 3);
  const explore = explorationPool(user, ctx, new Set(exploit));

  const feed = [];
  const used = new Set();
  let e = 0;
  while (feed.length < CONFIG.K) {
    const position = feed.length;
    let id;
    if (position >= o.protectTop && random() < eps && explore.length) {
      id = explore[Math.floor(random() * explore.length)];
    } else {
      while (e < exploit.length && used.has(exploit[e])) e++;
      id = exploit[e++];
```

```
    }
    if (id !== undefined && !used.has(id)) {
      used.add(id);
      feed.push(id);
    }
  }
  return feed;
}
```

2.5. Оцінювання та консольний інтерфейс

Модуль `metrics.js` реалізує метрики підрозділу 1.7. Функція `evaluate` перезасіває генератор псевдовипадкових чисел (тож алгоритми з випадковістю працюють в однакових умовах), будує стрічки для всіх користувачів, вимірює час функцією `performance.now()` і усереднює метрики. Обчислення nDCG наведено в лістингу 2.9; оскільки індекс позиції починається з нуля, у знаменнику використовується $\log_2(k + 2)$. Новизна обчислюється зі згладжуванням (likes + 1) / (m + 1), щоб уникнути логарифма нуля.

Лістинг 2.9 – Обчислення nDCG@K (metrics.js)

```
function ndcgAtK(feed, relevant, k = CONFIG.K) {
  let dcg = 0, idcg = 0;
  feed.forEach((id, i) => { if (relevant.has(id)) dcg += 1 / Math.log2(i + 2); });
  for (let i = 0; i < Math.min(k, relevant.size); i++) idcg += 1 / Math.log2(i +
2);
  return idcg ? dcg / idcg : 0;
}
```

Точка входу `main.js` підтримує команди, наведені в таблиці 2.2. Окремо реалізовано еталонний алгоритм-«оракул», який знає приховані параметри моделі даних і ранжує за істинною імовірністю вподобання (2.1). Він не бере участі в порівнянні, а визначає стелю якості, досяжну на цих даних.

Таблиця 2.2 – Команди консольного застосунку

Команда	**Дія**
node main.js	порівняння алгоритмів, перевірка стійкості та вибір алгоритму
node main.js params	дослідження впливу параметрів
node main.js seeds N	перевірка стійкості на N наборах даних

Команда	Дія
node main.js oracle	верхня межа якості
node main.js feed U	приклад стрічки користувача U

Фрагмент виведення стрічки для користувача №15 з інтересами music, travel і games наведено в лістингу 2.10; символом «✓» позначено публікації, які користувач насправді вподобав.

Лістинг 2.10 – Фрагмент виведення команди node main.js feed 15

```
--- Популярність ---
    Публікація #47   [tech, games, movies] ♥14, 95 год тому
    Публікація #112  [sport, food, music] ♥14, 94 год тому
  ✓ Публікація #92   [travel, music, games] ♥13, 178 год тому
--- Контентна фільтрація ---
    Публікація #81   [games, tech, travel]  ← бо ви любите: games, travel, tech
    Публікація #79   [games, travel]  ← бо ви любите: games, travel
    Публікація #113  [travel, games]  ← бо ви любите: travel, games
--- Гібридна зважена ---
    Публікація #210  [travel, tech, art] ♥12, 113 год тому
  ✓ Публікація #90   [games, politics, travel] ♥6, 5 год тому
    Публікація #81   [games, tech, travel] ♥7, 166 год тому
```

Приклад ілюструє відмінності алгоритмів: популярність показує однакові для всіх «хіти»; контентна фільтрація впевнено виявила інтерес до ігор і подорожей, але не розрізняє свіжість і якість (публікація #113 має вік 274 години й лише 3 вподобання); гібридна модель обирає публікації за інтересами, але свіжіші та популярніші.

2.6. Висновки до розділу 2

Розроблено модульний консольний застосунок мовою JavaScript обсягом близько 1000 рядків без сторонніх залежностей. Єдиний інтерфейс алгоритмів дозволив винести спільні кроки в окремий модуль, будувати складні алгоритми з простіших і додавати нові алгоритми одним рядком у реєстрі. Розроблено модель синтетичних даних з трьома прихованими чинниками вподобань і забезпечено відтворюваність експериментів та відсутність витоку даних. Кожен алгоритм реалізовано з параметрами, що

дозволяють дослідити його модифікації, і реалізовано модуль оцінювання з дев'ятьма метриками.

РОЗДІЛ 3. ПОРІВНЯЛЬНИЙ АНАЛІЗ ТА ВИБІР АЛГОРИТМУ

3.1. Умови експерименту та основні результати

Експеримент проведено на наборі даних з 300 публікацій, 100 користувачів і 1440 вподобань (зерно 42), довжина стрічки K = 10. Додатково всі алгоритми оцінено на десяти незалежно згенерованих наборах даних (зерна 1001–1010). Результати повністю відтворювані, за винятком часу виконання. Результати основного порівняння наведено в таблицях 3.1 і 3.2.

Таблиця 3.1 – Результати за метриками точності (K = 10, зерно 42)

Алгоритм	**Precision@10**	**Recall@10**	**nDCG@10**	**HitRate**
Хронологічний	0,016	0,054	0,034	0,16
Випадковий	0,013	0,055	0,027	0,13
За популярністю	0,020	0,069	0,045	0,19
Hacker News	0,020	0,059	0,040	0,18
Контентна фільтрація	0,030	0,102	0,060	0,27
Колаборативна (kNN)	0,018	0,062	0,035	0,15
Гібридна зважена	**0,044**	**0,161**	**0,094**	**0,39**
ε-greedy	0,039	0,137	0,084	0,35
Оракул (стеля якості)	0,055	0,209	0,133	0,45

Таблиця 3.2 – Результати за метриками «поза точністю» (зерно 42)

Алгоритм	**Покриття**	**ILD**	**Новизна, біт**	**Сер. вік, год**
Хронологічний	0,05	0,87	4,71	10
Випадковий	0,97	0,88	4,73	168
За популярністю	0,05	0,83	2,95	105
Hacker News	0,05	0,88	4,04	12
Контентна фільтрація	0,83	0,47	4,42	164
Колаборативна (kNN)	0,58	0,82	3,61	132
Гібридна зважена	0,45	0,75	3,54	78
ε-greedy	0,67	0,78	3,78	92

Для інтерпретації абсолютних значень врахуємо природні межі. Користувач має в середньому 2,9 прихованих вподобань серед 288 кандидатів, тому очікувана точність випадкової видачі становить 2,9 / 288 ≈ 0,01.

Отриманий результат 0,013 з цим узгоджується, що підтверджує коректність модуля оцінювання. Верхньою межею є результат оракула (nDCG = 0,133): він менший за 1, бо вподобання в моделі випадкові, і навіть найрелевантнішу публікацію користувач вподобає лише з імовірністю 30–50%. Саме з оракулом слід порівнювати алгоритми. Відносну якість кожного алгоритму щодо випадкової базової лінії та щодо оракула наведено в таблиці 3.3.

Таблиця 3.3 – Відносна якість алгоритмів за nDCG@10 (зерно 42)

Алгоритм	**nDCG@10**	**Відносно випадкового**	**Частка від оракула**
Випадковий	0,027	1,0×	20%
Хронологічний	0,034	1,3×	26%
Колаборативна (kNN)	0,035	1,3×	26%
Hacker News	0,040	1,5×	30%
За популярністю	0,045	1,7×	34%
Контентна фільтрація	0,060	2,2×	45%
ε-greedy	0,084	3,1×	63%
Гібридна зважена	0,094	3,5×	71%
Оракул (стеля)	0,133	4,9×	100%

3.2. Аналіз за метриками точності

Неперсоналізовані алгоритми показали низьку точність (nDCG від 0,027 до 0,045), оскільки основний чинник вподобань — збіг з персональними інтересами — їм недоступний. Хронологічний алгоритм лише незначно перевершив випадковий завдяки помірному впливу свіжості в моделі даних. Найточнішою серед неперсоналізованих виявилася популярність: вона корелює з якістю публікацій і, опосередковано, зі свіжістю. Формула Hacker News поступилася простій популярності, бо надто сильно віддає перевагу свіжості (середній вік показаних публікацій 12 годин), а на свіжих публікаціях ще не накопичилося достатньо вподобань для надійного сигналу якості.

Контентна фільтрація перевершила всі неперсоналізовані алгоритми, підтвердивши, що врахування інтересів є найважливішим чинником якості.

Водночас вона ігнорує якість і свіжість: середній вік рекомендованих публікацій (164 години) практично такий самий, як у випадкового алгоритму.

Колаборативна фільтрація (nDCG = 0,035) поступилася навіть популярності. Причини: розрідженість даних (за 11,5 вподобань у видимій історії перетини історій складаються з 1–3 публікацій, тому оцінки подібності «шумні»); малий розмір спільноти (лише кілька зі 100 користувачів мають такий самий набір інтересів); те, що теги в моделі ідеально описують інтереси, тож контентна фільтрація використовує цю інформацію безпосередньо, а колаборативна — лише опосередковано. Отже, результат демонструє відому залежність методу від обсягу даних, а не спростовує його цінність загалом.

Гібридна модель показала найкращі результати за всіма метриками точності: порівняно з контентною фільтрацією nDCG зріс на 57%, HitRate — з 0,27 до 0,39. Гібрид поєднує всі три «правди» даних: серед публікацій за інтересами він обирає якісні та свіжі. **ε-greedy** поступився гібриду, на якому побудований, на 11%: близько 1,2 з 10 позицій віддаються дослідженню. Офлайн-оцінювання принципово не може виміряти вигоду дослідження — нову інформацію для майбутніх рекомендацій, — тому ε-greedy слід оцінювати за метриками «поза точністю».

3.3. Аналіз за метриками різноманітності, новизни та свіжості

Метрики «поза точністю» (таблиця 3.2) виявляють недоліки, невидимі за точністю. Неперсоналізовані алгоритми, крім випадкового, мають **покриття** лише 0,05: усім показуються ті самі 15 публікацій, а 95% контенту не отримують аудиторії (ефект Метью). Персоналізовані алгоритми мають значно вище покриття, найвище — контентна фільтрація (0,83). ε-greedy підвищує покриття гібрида з 0,45 до 0,67, і в цьому його основна цінність.

Найнижча **різноманітність** у контентної фільтрації (ILD = 0,47, майже вдвічі менше, ніж у неперсоналізованих алгоритмів) — це кількісне підтвердження проблеми «бульбашки фільтрів». Гібридна модель з переранжуванням MMR, маючи найвищу точність, утримує різноманітність на

рівні 0,75. Найменша **новизна** в алгоритму популярності (2,95 біта), найбільша — у випадкового та хронологічного (4,7 біта); колаборативна фільтрація і гібрид мають помірну новизну через зміщення до популярного. **Свіжість** найвища в хронологічного алгоритму та формули Hacker News (10–12 годин), а гібрид показує публікації вдвічі свіжіші, ніж контентна фільтрація.

Отже, жоден алгоритм не є найкращим за всіма метриками, а гібридна модель досягає найкращого балансу.

3.4. Вплив параметрів алгоритмів

Вибрані результати дослідження параметрів наведено в таблиці 3.4.

Таблиця 3.4 – Вплив параметрів алгоритмів (зерно 42)

Алгоритм і варіант	**nDCG@10**	**Покриття**	**ILD**
Хронологічний, без обмеження / maxPerAuthor = 2	0,031 / 0,034	0,05	0,87
Випадковий, рівномірний / зважений A-Res	0,027 / 0,037	0,97 / 0,88	0,88 / 0,86
Популярність: лайки / частка / байєс / Вілсон	0,045 / 0,045 / 0,050 / 0,048	0,05	0,83–0,91
Hacker News G = 1,2 / 1,8 / 2,5; Reddit	0,042 / 0,040 / 0,037; 0,037	0,05	0,87
Контентна: бінарні ваги / TF-IDF	0,060 / 0,052	0,83 / 0,88	0,47 / 0,42
Колаборативна, Жаккар k = 5 / 15 / 40	0,031 / 0,035 / 0,048	0,77 / 0,58 / 0,43	0,81–0,82
Колаборативна, косинус IUF, k = 15	0,038	0,56	0,82
Гібридна: без MMR / λ = 0,85 / λ = 0,6	0,085 / 0,094 / 0,078	0,51 / 0,45 / 0,43	0,69 / 0,75 / 0,83
ε-greedy: ε = 0,1 / 0,2 / 0,4 (стале)	0,087 / 0,080 / 0,067	0,62 / 0,72 / 0,77	0,77 / 0,79 / 0,82

Обмеження на автора в хронологічній стрічці практично не впливає на точність, бо в моделі автор не впливає на вподобання; його цінність — у справедливості щодо авторів. Зважена випадкова вибірка точніша за рівномірну на 37% при високому покритті, тобто є вдалим компромісом між популярністю та шансом для нових публікацій. Для популярності статистично

обґрунтовані оцінки (байєсове середнє, межа Вілсона) підвищують точність на 7–11%, «гасячи» шум частки за малої кількості переглядів. Для часового згасання точнішим виявилося повільніше згасання (G = 1,2), а формула Reddit через логарифм популярності фактично стає хронологічною з невеликою поправкою; параметри таких формул мають відповідати швидкості старіння контенту платформи.

Несподіваним результатом є перевага бінарних ваг над TF-IDF у контентній фільтрації. Причина — множник $tf = 1 / |T(i)|$: він знижує вагу тегів у багатотемних публікаціях, хоча в моделі (як і в багатьох реальних системах) публікація з кількома збігами з інтересами вподобається з більшою, а не меншою імовірністю. Множник IDF майже не впливає, бо теги розподілені рівномірно. Отже, методи, перенесені з інших областей (TF-IDF створювався для текстів), потребують перевірки на даних конкретної задачі.

Точність колаборативної фільтрації монотонно зростає з кількістю сусідів (від 0,031 при $k = 5$ до 0,048 при $k = 40$): великий набір сусідів усереднює випадкові збіги, але рекомендації наближаються до загальної популярності, а покриття падає. IUF-зважування дає невелике покращення. Для гібридної моделі MMR з $\lambda = 0{,}85$ одночасно підвищує точність і різноманітність: користувачі мають 2–3 інтереси, і стрічка, що охоплює кілька з них, має більше шансів на влучання. Подальше зменшення λ до 0,6 витісняє релевантні публікації. Для ε-greedy зі зростанням ε точність монотонно падає, а покриття й різноманітність зростають; $\varepsilon = 0{,}1$ дає лише 7% втрати точності при збільшенні покриття на 38%.

3.5. Стійкість результатів

Щоб перевірити, чи не залежать висновки від особливостей одного набору даних, усі алгоритми оцінено на десяти незалежних наборах (таблиця 3.5).

Таблиця 3.5 – nDCG@10 на 10 незалежних наборах даних

Алгоритм	**Середнє**	**Ст. відхилення**	**Мінімум**	**Максимум**
Хронологічний	0,035	0,008	0,024	0,052
Випадковий	0,018	0,007	0,009	0,026
За популярністю	0,051	0,017	0,027	0,079
Hacker News	0,041	0,007	0,033	0,055
Контентна фільтрація	0,078	0,013	0,053	0,098
Колаборативна (kNN)	0,053	0,014	0,030	0,075
Гібридна зважена	**0,085**	0,014	0,060	0,102
ε-greedy	0,080	0,014	0,056	0,097
Оракул (стеля)	0,122	0,010	—	—

Загальний порядок алгоритмів зберігся, проте виявилося, що на основному наборі контентна фільтрація показала нетипово низький результат: у середньому її відставання від гібрида становить лише 9% (на основному наборі — 36%). Гібрид був найкращим на 8 наборах з 10, контентна фільтрація — на 2.

Значення nDCG трьох найкращих алгоритмів на кожному наборі наведено в таблиці 3.6. Видно, що на наборах 1004 і 1010 контентна фільтрація суттєво перевершила гібрид, тоді як ε-greedy поступається гібриду на кожному наборі.

Таблиця 3.6 – Значення nDCG@10 на кожному з 10 наборів даних

Зерно	**Контентна**	**Гібридна**	**ε-greedy**
1001	0,098	0,099	0,091
1002	0,087	0,091	0,087
1003	0,053	0,060	0,056
1004	0,074	0,067	0,061
1005	0,089	0,094	0,084
1006	0,081	0,088	0,081
1007	0,066	0,102	0,097
1008	0,067	0,092	0,088
1009	0,080	0,096	0,093
1010	0,087	0,066	0,060

Для коректного порівняння двох лідерів застосовано парний t-критерій Стьюдента до різниць nDCG на кожному наборі: такий підхід усуває вплив складності конкретного набору, на якому обидва алгоритми працюють гірше чи краще. Середня різниця становить 0,0073 при стандартному відхиленні різниць 0,0158:

$$t = \bar{d} / (s_d / \sqrt{N}) = 0{,}0073 / (0{,}0158 / \sqrt{10}) \approx 1{,}45. \quad (3.1)$$

Критичне значення для 9 ступенів свободи і рівня значущості 0,05 дорівнює 2,26, отже, **перевага гібридної моделі над контентною фільтрацією статистично незначуща**. Для порівняння, різниця між гібридом і ε-greedy (0,0055 при стандартному відхиленні 0,0020) дає $t \approx 8{,}55$: ціна дослідження стабільна і проявляється на кожному наборі. Отже, висновки з одного набору даних можуть бути спотвореними, а контентна фільтрація й гібридна модель утворюють групу лідерів за точністю.

3.6. Обґрунтування вибору алгоритму

Вибір алгоритму є багатокритеріальною задачею. Відповідно до мети роботи — **достатньо висока якість при мінімальній складності реалізації**, що характерно для невеликих команд і проєктів на початкових етапах, — сформульовано правило вибору:

1) основна метрика якості — середнє nDCG@10 на десяти наборах даних;
2) до розгляду допускаються алгоритми з якістю не менше 85% від найкращого; поріг відповідає приблизно одному стандартному відхиленню між наборами (близько 16% від якості лідерів), тож різниці в межах випадкових коливань не впливають на вибір;
3) серед допущених обирається алгоритм з найменшою складністю реалізації, а за рівної складності — найточніший.

Складність реалізації оцінено експертно з урахуванням обсягу коду, кількості параметрів, що потребують налаштування, і залежностей від інших компонентів. Результати застосування правила наведено в таблиці 3.7.

Таблиця 3.7 – Застосування правила вибору алгоритму

Алгоритм	**Сер. nDCG@10**	**Від найкращого**	**Рядків коду**	**Складність**
Гібридна зважена	0,085	100%	95 + компоненти	3
ε-greedy	0,080	94%	65 + гібрид	3
Контентна фільтрація	**0,078**	**91%**	**65**	**2**
Колаборативна (kNN)	0,053	62%	81	4
За популярністю	0,051	60%	50	1
Hacker News	0,041	48%	44	2
Хронологічний	0,035	41%	50	1
Випадковий	0,018	21%	52	1

Поріг $0{,}85 \cdot 0{,}085 \approx 0{,}072$ проходять три алгоритми, з яких найменшу складність має контентна фільтрація. Отже, **найкращим за співвідношенням якості та складності реалізації обрано контентну фільтрацію**. Це рішення автоматично відтворює функція `chooseAlgorithm` застосунку.

Вибір підтверджується такими аргументами: алгоритм досягає 91% якості найкращої моделі, а різниця статистично незначуща; він у 4,3 раза точніший за випадкову видачу і на 50% — за найкращий неперсоналізований алгоритм; його ядро займає близько 25 рядків і не потребує налаштування параметрів; він не залежить від інших користувачів і легко масштабується; нова публікація з тегами рекомендується одразу; покриття каталогу найвище серед персоналізованих алгоритмів; рекомендації пояснювані. Свідомо прийнятими обмеженнями є найнижча різноманітність стрічки, ігнорування якості та свіжості, залежність від якості тегів і холодний старт для нових користувачів, для яких слід використовувати запасний алгоритм популярності.

Для подальшого розвитку рекомендовано поетапно перетворювати контентну фільтрацію на гібридну модель: додати переранжування MMR, сигнали популярності (межа Вілсона) і свіжості, адаптивні ваги, а після

накопичення даних — колаборативний сигнал і дослідження з $\varepsilon \approx 0{,}1$, перевіряючи кожен етап тими самими метриками. Для систем з високими вимогами до точності перспективними є методи на основі моделей [10] і нейромережеве ранжування [4].

Результати слід інтерпретувати з урахуванням обмежень дослідження: синтетичні дані з ідеально точними тегами сприяють контентній фільтрації; невеликий обсяг даних шкодить колаборативній; офлайн-оцінювання не вимірює довгострокових ефектів; складність реалізації оцінено експертно; параметри алгоритмів не оптимізувалися систематично.

3.7. Висновки до розділу 3

Експериментально встановлено, що персоналізація є найважливішим чинником якості видачі: найкращі персоналізовані алгоритми точніші за найкращий неперсоналізований у 1,5–1,7 раза. Найвищу точність показала гібридна модель з MMR (nDCG = 0,085 в середньому, 70% від стелі якості) зі збалансованими різноманітністю, новизною і свіжістю. Контентна фільтрація досягла 91% якості гібрида, і різниця між ними статистично незначуща; колаборативна фільтрація на розріджених даних виявилася слабкою; ε-greedy стабільно поступається своїй основі в точності, але підвищує покриття каталогу. За сформульованим правилом обрано контентну фільтрацію як алгоритм з найкращим співвідношенням якості та складності реалізації.

ВИСНОВКИ

У курсовій роботі досліджено алгоритми видачі контенту користувачу, реалізовано їх у вигляді програмного застосунку та обґрунтовано вибір найкращого алгоритму. Отримано такі основні результати.

- Задачу видачі контенту сформульовано як задачу ранжування: побудови функції оцінки score(u, i) і відбору K об'єктів з найбільшими оцінками. Це формулювання стало основою теоретичного аналізу й архітектури застосунку.
- Розглянуто теоретичні основи восьми алгоритмів, що охоплюють усі основні класи: хронологічного, випадкового, за популярністю (з оцінками за кількістю вподобань, байєсовим середнім і межею Вілсона), з часовим згасанням (формули Hacker News і Reddit), контентної фільтрації, колаборативної фільтрації kNN, гібридної зваженої моделі з переранжуванням MMR і стратегії ε-greedy. Обґрунтовано систему з дев'яти метрик з основною метрикою nDCG@K.
- Розроблено модель синтетичних даних з трьома прихованими чинниками вподобань (інтереси, якість, свіжість) і реалізовано модульний консольний застосунок мовою JavaScript для Node.js обсягом близько 1000 рядків без сторонніх залежностей, з єдиним інтерфейсом алгоритмів і повною відтворюваністю результатів.
- Експериментально встановлено, що персоналізація є найважливішим чинником якості: найкращі персоналізовані алгоритми точніші за найкращий неперсоналізований у 1,5–1,7 раза. Найвищу точність показала гібридна модель (nDCG@10 = 0,085 в середньому на десяти наборах даних, близько 70% від теоретичної стелі 0,122). Колаборативна фільтрація на розріджених даних поступилася контентній, а ε-greedy,

поступаючись своїй основі в точності, підвищує покриття каталогу на 49%.

- Дослідження параметрів показало переваги статистично обґрунтованих оцінок популярності, бінарного зважування тегів над TF-IDF, великої кількості сусідів у колаборативній фільтрації та переранжування MMR, яке одночасно підвищує точність і різноманітність. Перевірка стійкості виявила, що перевага гібридної моделі над контентною фільтрацією статистично незначуща (t = 1,45 при критичному значенні 2,26).
- За правилом «найпростіший алгоритм серед тих, що досягають 85% якості найкращого» обрано **контентну фільтрацію**. Вона забезпечує 91% якості найкращої моделі, має ядро близько 25 рядків коду, не потребує налаштування параметрів, не має холодного старту для нового контенту й забезпечує пояснюваність рекомендацій. Для подолання її основного недоліку — низької різноманітності — запропоновано поетапний розвиток до гібридної моделі.

Мету роботи досягнуто, всі завдання виконано. Перспективними напрямами подальших досліджень є перевірка висновків на реальних даних, систематична оптимізація параметрів, реалізація матричної факторизації та онлайн-оцінювання стратегій дослідження.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Aggarwal C. C. Recommender Systems: The Textbook. Cham : Springer, 2016. 498 p.

2. Burke R. Hybrid Recommender Systems: Survey and Experiments. *User Modeling and User-Adapted Interaction*. 2002. Vol. 12, No. 4. P. 331–370.

3. Carbonell J., Goldstein J. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. *Proceedings of the 21st Annual International ACM SIGIR Conference*. 1998. P. 335–336.

4. Covington P., Adams J., Sargin E. Deep Neural Networks for YouTube Recommendations. *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16)*. 2016. P. 191–198.

5. Efraimidis P. S., Spirakis P. G. Weighted random sampling with a reservoir. *Information Processing Letters*. 2006. Vol. 97, No. 5. P. 181–185.

6. Herlocker J. L., Konstan J. A., Borchers A., Riedl J. An Algorithmic Framework for Performing Collaborative Filtering. *Proceedings of the 22nd Annual International ACM SIGIR Conference*. 1999. P. 230–237.

7. Jannach D., Zanker M., Felfernig A., Friedrich G. Recommender Systems: An Introduction. Cambridge : Cambridge University Press, 2010. 352 p.

8. Järvelin K., Kekäläinen J. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*. 2002. Vol. 20, No. 4. P. 422–446.

9. Knuth D. E. The Art of Computer Programming. Vol. 2: Seminumerical Algorithms. 3rd ed. Reading : Addison-Wesley, 1997. 762 p.

10. Koren Y., Bell R., Volinsky C. Matrix Factorization Techniques for Recommender Systems. *Computer*. 2009. Vol. 42, No. 8. P. 30–37.

11. Gomez-Uribe C. A., Hunt N. The Netflix Recommender System: Algorithms, Business Value, and Innovation. *ACM Transactions on Management Information Systems*. 2015. Vol. 6, No. 4. Article 13.

12. Linden G., Smith B., York J. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Computing*. 2003. Vol. 7, No. 1. P. 76–80.

13. Miller E. How Not To Sort By Average Rating. 2009. URL: https://www.evanmiller.org/how-not-to-sort-by-average-rating.html (дата звернення: 15.10.2026).

14. Pariser E. The Filter Bubble: What the Internet Is Hiding from You. New York : Penguin Press, 2011. 294 p.

15. Recommender Systems Handbook / ed. by F. Ricci, L. Rokach, B. Shapira. 3rd ed. New York : Springer, 2022. 1060 p.

16. Salihefendic A. How Reddit ranking algorithms work. *Hacking and Gonzo*. 2010. URL: https://medium.com/hacking-and-gonzo/how-reddit-ranking-algorithms-work-ef111e33d0d9 (дата звернення: 15.10.2026).

17. Salton G., Buckley C. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*. 1988. Vol. 24, No. 5. P. 513–523.

18. Sarwar B., Karypis G., Konstan J., Riedl J. Item-based collaborative filtering recommendation algorithms. *Proceedings of the 10th International Conference on World Wide Web (WWW '01)*. 2001. P. 285–295.

19. Sutton R. S., Barto A. G. Reinforcement Learning: An Introduction. 2nd ed. Cambridge : MIT Press, 2018. 552 p.

20. Wilson E. B. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*. 1927. Vol. 22, No. 158. P. 209–212.

21. Castells P., Hurley N., Vargas S. Novelty and Diversity in Recommender Systems. *Recommender Systems Handbook* / ed. by F. Ricci, L. Rokach, B. Shapira. 3rd ed. New York : Springer, 2022.

22. Node.js v22 Documentation. URL: https://nodejs.org/docs/latest/api/ (дата звернення: 15.10.2026).

23. ECMAScript Language Specification (ECMA-262). URL: https://tc39.es/ecma262/ (дата звернення: 15.10.2026).

24. Ziegler C.-N., McNee S. M., Konstan J. A., Lausen G. Improving Recommendation Lists Through Topic Diversification. *Proceedings of the 14th International Conference on World Wide Web (WWW '05)*. 2005. P. 22–32.

25. Shani G., Gunawardana A. Evaluating Recommendation Systems. *Recommender Systems Handbook* / ed. by F. Ricci, L. Rokach, B. Shapira, P. B. Kantor. New York : Springer, 2011. P. 257–297.

ДОДАТОК А. ЛІСТИНГ ПРОГРАМНОГО ЗАСТОСУНКУ

У додатку наведено повний вихідний код консольного застосунку. Для запуску потрібне середовище Node.js версії 14 або новішої. Файли слід розмістити відповідно до структури каталогів, наведеної в таблиці 2.1, і виконати в кореневому каталозі команду `node main.js`.

Лістинг А.1 – Файл config.js

```
'use strict';

module.exports = {
  SEED: 42,
  ITEMS: 300,
  USERS: 100,
  AUTHORS: 30,
  MAX_AGE_HOURS: 336,
  TEST_SHARE: 0.2,

  K: 10,

  TAGS: ['tech', 'sport', 'music', 'movies', 'games',
         'science', 'travel', 'food', 'politics', 'art'],
};
```

Лістинг А.2 – Файл lib/random.js

```
'use strict';

function mulberry32(seed) {
  return function () {
    seed |= 0;
    seed = (seed + 0x6D2B79F5) | 0;
    let t = Math.imul(seed ^ (seed >>> 15), 1 | seed);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

const state = { next: mulberry32(42) };

function reseed(seed) {
  state.next = mulberry32(seed);
}

function random() {
  return state.next();
}

function randInt(min, max) {
  return min + Math.floor(random() * (max - min + 1));
}
```

```
function sample(arr, n) {
  const copy = [...arr];
  const result = [];
  while (result.length < n && copy.length) {
    result.push(copy.splice(Math.floor(random() * copy.length), 1)[0]);
  }
  return result;
}

function shuffle(arr) {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

module.exports = { mulberry32, reseed, random, randInt, sample, shuffle };
```

Лістинг А.3 – Файл lib/math.js

```
'use strict';

function cosine(a, b) {
  let dot = 0, na = 0, nb = 0;
  for (const key in a) {
    na += a[key] * a[key];
    if (key in b) dot += a[key] * b[key];
  }
  for (const key in b) nb += b[key] * b[key];
  return na && nb ? dot / Math.sqrt(na * nb) : 0;
}

function jaccard(a, b) {
  let inter = 0;
  for (const x of a) if (b.has(x)) inter++;
  const union = a.size + b.size - inter;
  return union ? inter / union : 0;
}

function minMaxNormalize(map) {
  let min = Infinity, max = -Infinity;
  for (const v of map.values()) {
    if (v < min) min = v;
    if (v > max) max = v;
  }
  const range = max - min;
  const out = new Map();
  for (const [k, v] of map) out.set(k, range > 0 ? (v - min) / range : 0);
  return out;
}

function wilsonLowerBound(positive, n, z = 1.96) {
  if (n === 0) return 0;
  const p = positive / n;
  const z2 = z * z;
  const centre = p + z2 / (2 * n);
  const margin = z * Math.sqrt((p * (1 - p) + z2 / (4 * n)) / n);
  return (centre - margin) / (1 + z2 / n);
}
```

```
function mean(arr) {
  return arr.reduce((s, x) => s + x, 0) / arr.length;
}

function std(arr) {
  const m = mean(arr);
  return Math.sqrt(arr.reduce((s, x) => s + (x - m) ** 2, 0) / arr.length);
}

module.exports = { cosine, jaccard, minMaxNormalize, wilsonLowerBound, mean, std
};
```

Лістинг А.4 – Файл lib/ranking.js

```
'use strict';

const CONFIG = require('../config');

function candidates(user, ctx) {
  return ctx.items.filter(item => !user.train.has(item.id));
}

function rankScores(scores, k = CONFIG.K) {
  return [...scores.entries()]
    .sort((a, b) => b[1] - a[1] || a[0] - b[0])
    .slice(0, k)
    .map(([id]) => id);
}

function scoreCandidates(user, ctx, fn) {
  const scores = new Map();
  for (const item of candidates(user, ctx)) scores.set(item.id, fn(item));
  return scores;
}

module.exports = { candidates, rankScores, scoreCandidates };
```

Лістинг А.5 – Файл data/generator.js

```
'use strict';

const CONFIG = require('../config');
const { reseed, random, randInt, sample, shuffle } = require('../lib/random');

function generateItems() {
  const items = [];
  for (let id = 0; id < CONFIG.ITEMS; id++) {
    items.push({
      id,
      title: `Публікація #${id}`,
      authorId: randInt(0, CONFIG.AUTHORS - 1),
      tags: sample(CONFIG.TAGS, randInt(1, 3)),
      ageHours: Math.round(random() * CONFIG.MAX_AGE_HOURS),
      quality: random(),
      likes: 0,
      views: 0,
    });
```

```
  }
  return items;
}

function likeProbability(interests, item) {
  const overlap = item.tags.filter(t => interests.has(t)).length;
  let p = overlap > 0 ? 0.12 * overlap : 0.01;
  p *= 0.5 + item.quality;
  p *= 0.5 + 0.5 * (1 - item.ageHours / CONFIG.MAX_AGE_HOURS);
  return p;
}

function generateUsers(items) {
  const users = [];
  for (let id = 0; id < CONFIG.USERS; id++) {
    const interests = new Set(sample(CONFIG.TAGS, randInt(2, 3)));
    const liked = items.filter(item => random() < likeProbability(interests,
item))
                       .map(item => item.id);

    const mixed = shuffle(liked);
    const testSize = Math.max(1, Math.round(mixed.length * CONFIG.TEST_SHARE));
    users.push({
      id,
      interests,
      test: new Set(mixed.slice(0, testSize)),
      train: new Set(mixed.slice(testSize)),
    });
  }
  return users;
}

function buildContext(seed = CONFIG.SEED) {
  reseed(seed);
  const items = generateItems();
  const users = generateUsers(items);

  for (const user of users) {
    for (const id of user.train) items[id].likes++;
  }
  for (const item of items) {
    item.views = item.likes + item.likes * randInt(3, 12) + randInt(5, 60);
  }

  const tagDf = {};
  for (const item of items) {
    for (const tag of item.tags) tagDf[tag] = (tagDf[tag] || 0) + 1;
  }

  return {
    items,
    users,
    tagDf,
    maxLikes: Math.max(...items.map(i => i.likes)) || 1,
  };
}

module.exports = { buildContext, likeProbability };
```

Лістинг А.6 – Файл algorithms/index.js

```
'use strict';

module.exports = [
  require('./01-chronological'),
  require('./02-random'),
  require('./03-popularity'),
  require('./04-time-decay'),
  require('./05-content-based'),
  require('./06-collaborative'),
  require('./07-hybrid'),
  require('./08-epsilon-greedy'),
];
```

Лістинг А.7 – Файл algorithms/01-chronological.js

```
'use strict';

const CONFIG = require('../config');
const { candidates } = require('../lib/ranking');

const defaults = {
  maxPerAuthor: 2,
  cursor: null,
};

function scoreAll(user, ctx) {
  const scores = new Map();
  for (const item of candidates(user, ctx)) scores.set(item.id, -item.ageHours);
  return scores;
}

function recommend(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };

  let pool = candidates(user, ctx)
    .sort((a, b) => a.ageHours - b.ageHours || a.id - b.id);

  if (o.cursor !== null) pool = pool.filter(item => item.ageHours > o.cursor);

  const perAuthor = new Map();
  const feed = [];
  for (const item of pool) {
    const count = perAuthor.get(item.authorId) || 0;
    if (count >= o.maxPerAuthor) continue;
    perAuthor.set(item.authorId, count + 1);
    feed.push(item.id);
    if (feed.length === CONFIG.K) break;
  }
  return feed;
}

function nextCursor(feed, ctx) {
  return feed.length ? ctx.items[feed[feed.length - 1]].ageHours : null;
}

module.exports = {
  key: 'chronological',
```

```
  name: 'Хронологічний',
  difficulty: 1,
  personalized: false,
  defaults,
  scoreAll,
  recommend,
  nextCursor,
};
```

Лістинг А.8 – Файл algorithms/02-random.js

```
'use strict';

const CONFIG = require('../config');
const { random } = require('../lib/random');
const { candidates, rankScores } = require('../lib/ranking');

const defaults = {
  mode: 'uniform',
};

function uniformSample(arr, k) {
  const a = [...arr];
  const n = Math.min(k, a.length);
  for (let i = 0; i < n; i++) {
    const j = i + Math.floor(random() * (a.length - i));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a.slice(0, n);
}

function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const scores = new Map();
  for (const item of candidates(user, ctx)) {
    const u = random();
    if (o.mode === 'weighted') {
      const w = 1 + item.likes;
      scores.set(item.id, Math.pow(u, 1 / w));
    } else {
      scores.set(item.id, u);
    }
  }
  return scores;
}

function recommend(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  if (o.mode === 'uniform') {
    return uniformSample(candidates(user, ctx), CONFIG.K).map(item => item.id);
  }
  return rankScores(scoreAll(user, ctx, o));
}

module.exports = {
  key: 'random',
  name: 'Випадковий',
  difficulty: 1,
  personalized: false,
  defaults,
  scoreAll,
  recommend,
```

```
};
```

Лістинг А.9 – Файл algorithms/03-popularity.js

```
'use strict';

const { scoreCandidates, rankScores } = require('../lib/ranking');
const { wilsonLowerBound } = require('../lib/math');

const defaults = {
  mode: 'likes',
  bayesC: 50,
  z: 1.96,
};

function globalRatio(ctx) {
  let likes = 0, views = 0;
  for (const item of ctx.items) { likes += item.likes; views += item.views; }
  return views ? likes / views : 0;
}

function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const prior = globalRatio(ctx);

  return scoreCandidates(user, ctx, item => {
    switch (o.mode) {
      case 'ratio':
        return item.views ? item.likes / item.views : 0;
      case 'bayes':
        return (item.likes + o.bayesC * prior) / (item.views + o.bayesC);
      case 'wilson':
        return wilsonLowerBound(item.likes, item.views, o.z);
      case 'likes':
      default:
        return item.likes;
    }
  });
}

function recommend(user, ctx, opts = {}) {
  return rankScores(scoreAll(user, ctx, opts));
}

module.exports = {
  key: 'popularity',
  name: 'Популярність',
  difficulty: 1,
  personalized: false,
  defaults,
  scoreAll,
  recommend,
  globalRatio,
};
```

Лістинг А.10 – Файл algorithms/04-time-decay.js

```
'use strict';
```

```
const { scoreCandidates, rankScores } = require('../lib/ranking');

const defaults = {
  mode: 'hn',
  gravity: 1.8,
};

function hackerNewsScore(likes, ageHours, gravity) {
  return likes / Math.pow(ageHours + 2, gravity);
}

function redditHotScore(likes, ageHours) {
  const s = likes;
  const order = Math.log10(Math.max(Math.abs(s), 1));
  const sign = s > 0 ? 1 : s < 0 ? -1 : 0;
  const ageSeconds = ageHours * 3600;
  return sign * order - ageSeconds / 45000;
}

function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  return scoreCandidates(user, ctx, item =>
    o.mode === 'reddit'
      ? redditHotScore(item.likes, item.ageHours)
      : hackerNewsScore(item.likes, item.ageHours, o.gravity));
}

function recommend(user, ctx, opts = {}) {
  return rankScores(scoreAll(user, ctx, opts));
}

module.exports = {
  key: 'timeDecay',
  name: 'Hacker News',
  difficulty: 2,
  personalized: false,
  defaults,
  scoreAll,
  recommend,
  hackerNewsScore,
  redditHotScore,
};
```

Лістинг А.11 – Файл algorithms/05-content-based.js

```
'use strict';

const { candidates, rankScores } = require('../lib/ranking');
const { cosine } = require('../lib/math');

const defaults = {
  useIdf: false,
};

function idf(tag, ctx) {
  return Math.log(ctx.items.length / (ctx.tagDf[tag] || 1));
}

function itemVector(item, ctx, useIdf) {
  const v = {};
```

```
  const tf = 1 / item.tags.length;
  for (const tag of item.tags) v[tag] = useIdf ? tf * idf(tag, ctx) : 1;
  return v;
}

function buildProfile(user, ctx, useIdf = defaults.useIdf) {
  const profile = {};
  for (const id of user.train) {
    const v = itemVector(ctx.items[id], ctx, useIdf);
    for (const tag in v) profile[tag] = (profile[tag] || 0) + v[tag];
  }
  return profile;
}

function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const profile = buildProfile(user, ctx, o.useIdf);
  const scores = new Map();
  for (const item of candidates(user, ctx)) {
    scores.set(item.id, cosine(profile, itemVector(item, ctx, o.useIdf)));
  }
  return scores;
}

function recommend(user, ctx, opts = {}) {
  return rankScores(scoreAll(user, ctx, opts));
}

function explain(user, item, ctx) {
  const profile = buildProfile(user, ctx);
  const v = itemVector(item, ctx, defaults.useIdf);
  return Object.keys(v)
    .map(tag => ({ tag, contribution: (profile[tag] || 0) * v[tag] }))
    .filter(x => x.contribution > 0)
    .sort((a, b) => b.contribution - a.contribution)
    .map(x => x.tag);
}

module.exports = {
  key: 'contentBased',
  name: 'Контентна фільтрація',
  difficulty: 2,
  personalized: true,
  defaults,
  scoreAll,
  recommend,
  buildProfile,
  itemVector,
  explain,
};
```

Лістинг А.12 – Файл algorithms/06-collaborative.js

```
'use strict';

const { candidates, rankScores } = require('../lib/ranking');
const { jaccard } = require('../lib/math');

const defaults = {
  similarity: 'jaccard',
  neighbours: 15,
```

```
  minSimilarity: 0.01,
};


function inverseUserFrequency(ctx) {
  const iuf = new Map();
  const U = ctx.users.length;
  for (const item of ctx.items) iuf.set(item.id, Math.log(U / (item.likes + 1)));
  return iuf;
}


function cosineIUF(a, b, iuf) {
  let dot = 0, na = 0, nb = 0;
  for (const id of a) {
    const w = iuf.get(id) ** 2;
    na += w;
    if (b.has(id)) dot += w;
  }
  for (const id of b) nb += iuf.get(id) ** 2;
  return na && nb ? dot / Math.sqrt(na * nb) : 0;
}


function findNeighbours(user, ctx, o) {
  const iuf = o.similarity === 'cosine' ? inverseUserFrequency(ctx) : null;
  return ctx.users
    .filter(v => v.id !== user.id)
    .map(v => ({
      user: v,
      sim: o.similarity === 'cosine'
        ? cosineIUF(user.train, v.train, iuf)
        : jaccard(user.train, v.train),
    }))
    .filter(n => n.sim > o.minSimilarity)
    .sort((a, b) => b.sim - a.sim || a.user.id - b.user.id)
    .slice(0, o.neighbours);
}


function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const neighbours = findNeighbours(user, ctx, o);
  const scores = new Map();


  if (neighbours.length === 0) {
    for (const item of candidates(user, ctx)) {
      scores.set(item.id, item.likes / ctx.maxLikes);
    }
    return scores;
  }


  const simSum = neighbours.reduce((s, n) => s + n.sim, 0);
  const raw = new Map();
  for (const { user: v, sim } of neighbours) {
    for (const id of v.train) raw.set(id, (raw.get(id) || 0) + sim);
  }
  for (const item of candidates(user, ctx)) {
    scores.set(item.id, (raw.get(item.id) || 0) / simSum);
  }
  return scores;
}


function recommend(user, ctx, opts = {}) {
  return rankScores(scoreAll(user, ctx, opts));
}
```

```
module.exports = {
  key: 'collaborative',
  name: 'Колаборативна (kNN)',
  difficulty: 4,
  personalized: true,
  defaults,
  scoreAll,
  recommend,
  findNeighbours,
};
```

Лістинг А.13 – Файл algorithms/07-hybrid.js

```
'use strict';

const CONFIG = require('../config');
const { candidates } = require('../lib/ranking');
const { minMaxNormalize, jaccard } = require('../lib/math');
const contentBased = require('./05-content-based');
const collaborative = require('./06-collaborative');
const popularity = require('./03-popularity');

const defaults = {
  weights: { content: 0.45, collaborative: 0.2, popularity: 0.2, freshness: 0.15
},
  halfLifeHours: 96,
  coldStartHistory: 5,
  mmrLambda: 0.85,
  useMMR: true,
};

function adaptWeights(user, o) {
  const w = { ...o.weights };
  const n = user.train.size;
  if (n < o.coldStartHistory) {
    const trust = n / o.coldStartHistory;
    const moved = (w.content + w.collaborative) * (1 - trust);
    w.content *= trust;
    w.collaborative *= trust;
    w.popularity += moved;
  }
  return w;
}

function scoreAll(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts, weights: { ...defaults.weights,
...(opts.weights || {}) } };
  const w = adaptWeights(user, o);

  const signals = {
    content: minMaxNormalize(contentBased.scoreAll(user, ctx)),
    collaborative: minMaxNormalize(collaborative.scoreAll(user, ctx)),
    popularity: minMaxNormalize(popularity.scoreAll(user, ctx, { mode: 'wilson'
})),
  };

  const scores = new Map();
  for (const item of candidates(user, ctx)) {
    const freshness = Math.exp(-Math.LN2 * item.ageHours / o.halfLifeHours);
    scores.set(item.id,
      w.content * signals.content.get(item.id) +
      w.collaborative * signals.collaborative.get(item.id) +
```

```
      w.popularity * signals.popularity.get(item.id) +
      w.freshness * freshness);
  }
  return scores;
}

function mmrRerank(scores, ctx, lambda, k = CONFIG.K) {
  const pool = [...scores.entries()]
    .sort((a, b) => b[1] - a[1] || a[0] - b[0])
    .slice(0, k * 5);

  const tags = new Map(pool.map(([id]) => [id, new Set(ctx.items[id].tags)]));

  const redundancy = new Map(pool.map(([id]) => [id, 0]));
  const selected = [];

  while (selected.length < k && pool.length) {
    let bestIdx = 0, bestValue = -Infinity;
    pool.forEach(([id, score], idx) => {
      const value = lambda * score - (1 - lambda) * redundancy.get(id);
      if (value > bestValue) { bestValue = value; bestIdx = idx; }
    });
    const [chosen] = pool.splice(bestIdx, 1)[0];
    selected.push(chosen);
    for (const [id] of pool) {
      const sim = jaccard(tags.get(id), tags.get(chosen));
      if (sim > redundancy.get(id)) redundancy.set(id, sim);
    }
  }
  return selected;
}

function recommend(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const scores = scoreAll(user, ctx, opts);
  return mmrRerank(scores, ctx, o.useMMR ? o.mmrLambda : 1);
}

module.exports = {
  key: 'hybrid',
  name: 'Гібридна зважена',
  difficulty: 3,
  personalized: true,
  defaults,
  scoreAll,
  recommend,
  adaptWeights,
  mmrRerank,
};
```

Лістинг А.14 – Файл algorithms/08-epsilon-greedy.js

```
'use strict';

const CONFIG = require('../config');
const { random } = require('../lib/random');
const { candidates } = require('../lib/ranking');
const hybrid = require('./07-hybrid');

const defaults = {
  epsilon: 0.2,
```

```
  epsilonMin: 0.05,
  decay: true,
  tau: 40,
  protectTop: 2,
};

function currentEpsilon(user, o) {
  if (!o.decay) return o.epsilon;
  return Math.max(o.epsilonMin, o.epsilon / (1 + user.train.size / o.tau));
}

function explorationPool(user, ctx, exploitSet) {
  const pool = candidates(user, ctx)
    .filter(item => !exploitSet.has(item.id))
    .sort((a, b) => a.likes - b.likes || a.id - b.id);

  return pool.slice(0, Math.max(1, Math.ceil(pool.length / 2))).map(i => i.id);
}

function recommend(user, ctx, opts = {}) {
  const o = { ...defaults, ...opts };
  const eps = currentEpsilon(user, o);

  const scores = hybrid.scoreAll(user, ctx);
  const exploit = hybrid.mmrRerank(scores, ctx, hybrid.defaults.mmrLambda,
CONFIG.K * 3);
  const explore = explorationPool(user, ctx, new Set(exploit));

  const feed = [];
  const used = new Set();
  let e = 0;
  while (feed.length < CONFIG.K) {
    const position = feed.length;
    let id;
    if (position >= o.protectTop && random() < eps && explore.length) {
      id = explore[Math.floor(random() * explore.length)];
    } else {
      while (e < exploit.length && used.has(exploit[e])) e++;
      id = exploit[e++];
    }
    if (id !== undefined && !used.has(id)) {
      used.add(id);
      feed.push(id);
    }
  }
  return feed;
}

module.exports = {
  key: 'epsilonGreedy',
  name: 'ε-greedy',
  difficulty: 3,
  personalized: true,
  defaults,
  recommend,
  currentEpsilon,
};
```

Лістинг А.15 – Файл metrics.js

```
'use strict';
```

```
const CONFIG = require('./config');
const { reseed } = require('./lib/random');
const { jaccard } = require('./lib/math');

const now = () => (typeof performance !== 'undefined' ? performance.now() :
Date.now());

function ndcgAtK(feed, relevant, k = CONFIG.K) {
  let dcg = 0, idcg = 0;
  feed.forEach((id, i) => { if (relevant.has(id)) dcg += 1 / Math.log2(i + 2); });
  for (let i = 0; i < Math.min(k, relevant.size); i++) idcg += 1 / Math.log2(i +
2);
  return idcg ? dcg / idcg : 0;
}

function intraListDiversity(feed, ctx) {
  let sum = 0, pairs = 0;
  for (let i = 0; i < feed.length; i++) {
    for (let j = i + 1; j < feed.length; j++) {
      const a = new Set(ctx.items[feed[i]].tags);
      const b = new Set(ctx.items[feed[j]].tags);
      sum += 1 - jaccard(a, b);
      pairs++;
    }
  }
  return pairs ? sum / pairs : 0;
}

function novelty(feed, ctx) {
  const U = ctx.users.length;
  const values = feed.map(id => -Math.log2((ctx.items[id].likes + 1) / (U + 1)));
  return values.reduce((s, x) => s + x, 0) / feed.length;
}

function evaluate(algo, ctx, opts = {}, label = algo.name) {
  reseed(CONFIG.SEED + 1);

  const t0 = now();
  const feeds = ctx.users.map(u => algo.recommend(u, ctx, opts));
  const timeMs = now() - t0;

  const acc = { precision: 0, recall: 0, ndcg: 0, hit: 0, ild: 0, novelty: 0, age:
0 };
  const shown = new Set();

  ctx.users.forEach((user, idx) => {
    const feed = feeds[idx];
    const hits = feed.filter(id => user.test.has(id)).length;
    acc.precision += hits / CONFIG.K;
    acc.recall += hits / user.test.size;
    acc.ndcg += ndcgAtK(feed, user.test);
    if (hits > 0) acc.hit++;
    acc.ild += intraListDiversity(feed, ctx);
    acc.novelty += novelty(feed, ctx);
    acc.age += feed.reduce((s, id) => s + ctx.items[id].ageHours, 0) /
feed.length;
    feed.forEach(id => shown.add(id));
  });

  const n = ctx.users.length;
  return {
```

```
    name: label,
    precision: acc.precision / n,
    recall: acc.recall / n,
    ndcg: acc.ndcg / n,
    hitRate: acc.hit / n,
    coverage: shown.size / ctx.items.length,
    ild: acc.ild / n,
    novelty: acc.novelty / n,
    avgAge: acc.age / n,
    timeMs,
    difficulty: algo.difficulty,
  };
}

function toRow(r) {
  return {
    'Алгоритм': r.name,
    'Precision': +r.precision.toFixed(3),
    'Recall': +r.recall.toFixed(3),
    'nDCG': +r.ndcg.toFixed(3),
    'HitRate': +r.hitRate.toFixed(2),
    'Покриття': +r.coverage.toFixed(2),
    'ILD': +r.ild.toFixed(2),
    'Новизна': +r.novelty.toFixed(2),
    'Вік, год': Math.round(r.avgAge),
    'Час, мс': +r.timeMs.toFixed(1),
    'Складн.': r.difficulty,
  };
}

module.exports = { evaluate, toRow, ndcgAtK, intraListDiversity, novelty };
```

Лістинг А.16 – Файл main.js

```
'use strict';

const CONFIG = require('./config');
const { buildContext } = require('./data/generator');
const { evaluate, toRow } = require('./metrics');
const { reseed } = require('./lib/random');
const { mean, std } = require('./lib/math');
const ALGORITHMS = require('./algorithms');

const byKey = Object.fromEntries(ALGORITHMS.map(a => [a.key, a]));

function header(title) {
  console.log('\n' + '='.repeat(78));
  console.log(' ' + title);
  console.log('='.repeat(78));
}

function runComparison(ctx) {
  header('ПОРІВНЯННЯ АЛГОРИТМІВ ВИДАЧІ КОНТЕНТУ');
  const totalLikes = ctx.users.reduce((s, u) => s + u.train.size + u.test.size,
0);
  console.log(`Публікацій: ${CONFIG.ITEMS}, користувачів: ${CONFIG.USERS}, ` +
              `авторів: ${CONFIG.AUTHORS}, вподобань: ${totalLikes}, K =
${CONFIG.K}`);

  const results = ALGORITHMS.map(a => evaluate(a, ctx));
  console.table(results.map(toRow));
```

```
  return results;
}


function chooseAlgorithm(meanNdcg) {
  const rows = ALGORITHMS.map(a => ({ name: a.name, difficulty: a.difficulty,
ndcg: meanNdcg[a.name] }));
  const best = [...rows].sort((a, b) => b.ndcg - a.ndcg)[0];
  const chosen = rows
    .filter(r => r.ndcg >= best.ndcg * 0.85)
    .sort((a, b) => a.difficulty - b.difficulty || b.ndcg - a.ndcg)[0];


  console.log(`\nНайвища середня якість (nDCG@${CONFIG.K}): ${best.name} —
${best.ndcg.toFixed(3)}`);
  console.log(`ОБРАНО — найпростіший алгоритм з якістю >= 85% від найкращої:
${chosen.name} — ` +
              `${chosen.ndcg.toFixed(3)} (${Math.round(chosen.ndcg / best.ndcg *
100)}% від найкращого, ` +
              `складність ${chosen.difficulty} з 5)`);
  return chosen;
}


function runParams(ctx) {
  header('ВПЛИВ ПАРАМЕТРІВ АЛГОРИТМІВ');


  const experiments = [
    ['Хронологічний: обмеження на автора', 'chronological', [
      ['без обмеження', { maxPerAuthor: Infinity }],
      ['maxPerAuthor = 2', { maxPerAuthor: 2 }],
      ['maxPerAuthor = 1', { maxPerAuthor: 1 }],
    ]],
    ['Випадковий: режим вибірки', 'random', [
      ['рівномірна', { mode: 'uniform' }],
      ['зважена (A-Res)', { mode: 'weighted' }],
    ]],
    ['Популярність: спосіб оцінки', 'popularity', [
      ['лайки', { mode: 'likes' }],
      ['частка лайків', { mode: 'ratio' }],
      ['байєсове середнє', { mode: 'bayes' }],
      ['межа Вілсона', { mode: 'wilson' }],
    ]],
    ['Часове згасання: формула і гравітація', 'timeDecay', [
      ['HN, G = 1.2', { mode: 'hn', gravity: 1.2 }],
      ['HN, G = 1.8', { mode: 'hn', gravity: 1.8 }],
      ['HN, G = 2.5', { mode: 'hn', gravity: 2.5 }],
      ['Reddit Hot', { mode: 'reddit' }],
    ]],
    ['Контентна фільтрація: зважування тегів', 'contentBased', [
      ['бінарні ваги', { useIdf: false }],
      ['TF-IDF', { useIdf: true }],
    ]],
    ['Колаборативна: міра подібності і k', 'collaborative', [
      ['Жаккар, k = 5', { similarity: 'jaccard', neighbours: 5 }],
      ['Жаккар, k = 15', { similarity: 'jaccard', neighbours: 15 }],
      ['Жаккар, k = 40', { similarity: 'jaccard', neighbours: 40 }],
      ['косинус IUF, k = 15', { similarity: 'cosine', neighbours: 15 }],
    ]],
    ['Гібридна: переранжування MMR', 'hybrid', [
      ['без MMR', { useMMR: false }],
      ['MMR, λ = 0.85', { useMMR: true, mmrLambda: 0.85 }],
      ['MMR, λ = 0.6', { useMMR: true, mmrLambda: 0.6 }],
      ['лише контент', { useMMR: false, weights: { content: 1, collaborative: 0,
popularity: 0, freshness: 0 } }],
    ]],
    ['ε-greedy: імовірність дослідження', 'epsilonGreedy', [
      ['ε = 0.1, без згасання', { epsilon: 0.1, decay: false }],
```

```
      ['ε = 0.2, без згасання', { epsilon: 0.2, decay: false }],
      ['ε = 0.2, зі згасанням', { epsilon: 0.2, decay: true }],
      ['ε = 0.4, без згасання', { epsilon: 0.4, decay: false }],
    ]],
  ];

  const all = {};
  for (const [title, key, variants] of experiments) {
    console.log(`\n--- ${title} ---`);
    const rows = variants.map(([label, opts]) => evaluate(byKey[key], ctx, opts,
label));
    console.table(rows.map(toRow).map(r => {
      delete r['Складн.'];
      return r;
    }));
    all[key] = rows;
  }
  return all;
}

function runSeeds(count) {
  header(`СТІЙКІСТЬ РЕЗУЛЬТАТІВ: ${count} НАБОРІВ ДАНИХ`);
  const table = {};
  for (let s = 1; s <= count; s++) {
    const ctx = buildContext(1000 + s);
    for (const a of ALGORITHMS) {
      (table[a.name] ||= []).push(evaluate(a, ctx).ndcg);
    }
  }
  const rows = Object.entries(table).map(([name, values]) => ({
    'Алгоритм': name,
    'nDCG сер.': +mean(values).toFixed(3),
    'Ст. відх.': +std(values).toFixed(3),
    'Мін': +Math.min(...values).toFixed(3),
    'Макс': +Math.max(...values).toFixed(3),
  }));
  console.table(rows);

  const wins = {};
  for (let i = 0; i < count; i++) {
    const winner = Object.entries(table).sort((a, b) => b[1][i] - a[1][i])[0][0];
    wins[winner] = (wins[winner] || 0) + 1;
  }
  console.log('Перемоги за nDCG:', wins);

  const meanNdcg = Object.fromEntries(Object.entries(table).map(([n, v]) => [n,
mean(v)]));
  const chosen = chooseAlgorithm(meanNdcg);
  return { rows, wins, table, chosen };
}

const { likeProbability } = require('./data/generator');
const { candidates, rankScores } = require('./lib/ranking');

const oracle = {
  key: 'oracle',
  name: 'Оракул (стеля якості)',
  difficulty: 0,
  recommend(user, ctx) {
    const scores = new Map();
    for (const item of candidates(user, ctx)) {
      scores.set(item.id, likeProbability(user.interests, item));
    }
    return rankScores(scores);
```

```
  },
};


function runOracle(count = 10) {
  header('ВЕРХНЯ МЕЖА ЯКОСТІ (ОРАКУЛ)');
  const values = [];
  for (let s = 1; s <= count; s++) values.push(evaluate(oracle, buildContext(1000
+ s)).ndcg);
  const single = evaluate(oracle, buildContext());
  console.table([toRow(single)]);
  console.log(`nDCG оракула на ${count} наборах: ${mean(values).toFixed(3)} ±
${std(values).toFixed(3)}`);
}


function printFeed(userId, ctx) {
  const user = ctx.users[userId];
  if (!user) {
    console.log(`Користувача №${userId} не існує (0..${ctx.users.length - 1}).`);
    return;
  }
  header(`СТРІЧКА КОРИСТУВАЧА №${user.id}`);
  console.log(`Справжні інтереси (прихований параметр):
${[...user.interests].join(', ')}`);
  console.log(`Вподобань в історії: ${user.train.size}, приховано для перевірки:
${user.test.size}\n`);


  for (const algo of ALGORITHMS) {
    reseed(CONFIG.SEED + 1);
    const feed = algo.recommend(user, ctx).slice(0, 5);
    console.log(`--- ${algo.name} ---`);
    for (const id of feed) {
      const it = ctx.items[id];
      const mark = user.test.has(id) ? '✓' : ' ';
      let line = `  ${mark} ${it.title.padEnd(16)} [${it.tags.join(', ')}] ` +
                 `♥${it.likes}, ${it.ageHours} год тому, автор ${it.authorId}`;
      if (algo.key === 'contentBased') {
        line += `  ← бо ви любите: ${byKey.contentBased.explain(user, it,
ctx).join(', ')}`;
      }
      console.log(line);
    }
    console.log('');
  }


  const chrono = byKey.chronological;
  const page1 = chrono.recommend(user, ctx);
  const cursor = chrono.nextCursor(page1, ctx);
  const page2 = chrono.recommend(user, ctx, { cursor });
  console.log(`Хронологічна стрічка, сторінка 2 (курсор = ${cursor} год): ` +
              page2.slice(0, 5).map(id => `#${id}`).join(', ') + ' ...');
  console.log('✓ — публікація, яку користувач насправді вподобав (влучання).');
}


function main() {
  const args = typeof process !== 'undefined' ? process.argv.slice(2) : [];
  const command = args[0] || 'compare';
  const ctx = buildContext();


  switch (command) {
    case 'compare': runComparison(ctx); runSeeds(10); break;
    case 'params':  runParams(ctx); break;
    case 'seeds':   runSeeds(Number(args[1] || 10)); break;
    case 'feed':    printFeed(Number(args[1] ?? 0), ctx); break;
    case 'oracle':  runOracle(10); break;
```

```
    case 'all':
      runComparison(ctx);
      runParams(ctx);
      runSeeds(10);
      runOracle(10);
      printFeed(15, ctx);
      break;
    default:
      console.log('Невідома команда. Доступні: compare, params, seeds N, feed N,
all');
  }
}

if (require.main === module) main();

module.exports = { runComparison, runParams, runSeeds, runOracle, printFeed,
chooseAlgorithm };
```